ASESII 24A02 Project-Based Quiz 1

Ridge, Lasso, and Regularization for Neural Features

Group 04
Nguyen Thi Yen Nhi (24125071)
Nguyen Khang Thinh (24125104)
Nguyen Van Tinh (24125106)
Nguyen Le Quoc Huy (24125100)

2026-07-17

Contents

Authorship and contribution statement

We confirm that every group member can explain the submitted analysis. AI use is reported in `Group04_AI_Log.csv`, and the group checked every AI-assisted item used in this report.

Student	ID	Main contributions	Verification performed
Nguyen Thi Yen Nhi	24125071	[Work]	[Check]
Nguyen Khang Thinh	24125104	[Work]	[Check]
Nguyen Van Tinh	24125106	[Work]	[Check]
Nguyen Le Quoc Huy	24125100	[Work]	[Check]

Abstract

This study aims to predict pre-operative log prostate-specific antigen (`lpsa`) from clinical measurements, comparing models on prediction error, coefficient stability, and interpretability under multicollinearity. We evaluated OLS, Ridge, Lasso, and Elastic Net regressions using five-fold cross-validation on original linear predictors and fixed random ReLU features. Cross-validation initially favored Lasso (CV-MSE = 0.5790) for its sparse selection of five variables. However, holdout evaluation revealed that the OLS model achieved the lowest test RMSE (0.6713) and MAE (0.5736). Furthermore, original linear features consistently outperformed the nonlinear ReLU representations across all models. Despite the OLS superiority in pure holdout accuracy, we recommend the lasso-dominant elastic net ($\alpha = 0.9$) for clinical deployment, as it provides a parsimonious, highly interpretable model that isolates key biological drivers of PSA with only a modest trade-off in error.

Reproducible setup

```
required_packages <- c("tidyverse", "glmnet", "broom", "knitr", "car")
missing_packages <- required_packages[
  !vapply(required_packages, requireNamespace, logical(1), quietly = TRUE)
]
if (length(missing_packages)) {
  stop("Install the required packages before rendering: ",
       paste(missing_packages, collapse = ", "))
}

library(tidyverse)
library(glmnet)
library(broom)
library(knitr)
library(car)

# `params` exists only while knitting; the fallback
# keeps interactive chunk runs working.
has_params <- exists("params") && !is.null(params$group_number)
group_number <- if (has_params) as.integer(params$group_number) else 4L
if (length(group_number) != 1L || is.na(group_number) || group_number < 1L) {
  stop("Set params$group_number to your assigned positive group number.")
}

seeds <- list(
  split = 240200L + group_number,
  cv = 240300L + group_number,
  features = 240400L + group_number
)
```

Shared helper functions

1 Problem 1. Prediction design and leakage control

1.1 1A. Target and predictors

```
# The brief assigns prostate.csv to even-numbered groups.
stopifnot(group_number %% 2L == 0L)

config <- list(
  file = "prostate.csv",
  response = "lpsa",
  excluded = "lpsa"
```

```
)
data_raw <- read.csv(find_exam_data(config$file), check.names = FALSE)
predictors <- setdiff(names(data_raw), config$excluded)
analysis_data <- data_raw[, c(config$response, predictors), drop = FALSE]

if (anyNA(analysis_data)) stop("Declare a training-only missing-value plan.")
if (!all(vapply(analysis_data, is.numeric, logical(1)))) {
  stop("The assigned response and predictors must be numeric.")
}
```

Dataset overview. The `prostate.csv` file originates from the study by Stamey et al. (1989), which examined the relationship between prostate-specific antigen (PSA) levels and several clinical and histological measurements in men with clinically localized prostate cancer who underwent radical prostatectomy. The dataset contains 97 observations and 9 variables - one continuous response (`lpsa`) and 8 candidate predictors covering tumour characteristics, patient demographics, and biopsy summaries.

Response. `lpsa` is the natural logarithm of prostate-specific antigen (ng/mL), a continuous serum marker measured before surgery. It is modelled on the log scale, so every coefficient is read as a change in log PSA per one standard deviation of a standardized predictor.

Unit of observation and study population. One row is one male patient with clinically localized prostate cancer who underwent radical prostatectomy in the study of Stamey et al. (1989). The analysis file contains 97 complete patients. Any conclusion therefore generalizes only to comparable prostatectomy candidates, not to unscreened men in the general population.

Prediction objective. Predict pre-operative `lpsa` from routinely available clinical and biopsy measurements, and compare how OLS, ridge, and lasso trade prediction error against coefficient stability and interpretability when several predictors are correlated.

Candidate predictors. All 8 remaining variables are retained as candidate predictors. They are drawn from the clinical and pathological records described in Stamey et al. (1989):

- `lcavol` – log of cancer volume (log cm^3), estimated by ultrasound before surgery. Larger tumours are expected to produce more PSA.
- `lweight` – log of total prostate weight (log g), measured at prostatectomy. Reflects overall prostate size, including both cancerous and non-cancerous tissue.
- `age` – patient age at the time of diagnosis (years).
- `lbph` – log of the amount of benign prostatic hyperplasia (log cm^2). BPH is non-cancerous prostate enlargement and can independently elevate PSA.
- `svi` – seminal vesicle invasion (binary: 1 = present, 0 = absent), indicating whether cancer has spread beyond the prostate into the seminal vesicles. This is a known marker of advanced disease.
- `lcp` – log of capsular penetration (log cm), measuring the extent to which cancer has invaded through the prostate capsule. Together with `svi`, it captures local disease progression.
- `gleason` – combined Gleason score (integer, ranging from 6 to 9 in this sample), the standard histological grading system for prostate cancer aggressiveness assigned by a pathologist from biopsy tissue.

- `pgg45` – percentage of Gleason grades 4 or 5 in the biopsy specimen (percent, 0-100). Higher values indicate a larger proportion of poorly-differentiated, aggressive tumour tissue.

Exclusions. The project brief requires only that `lpsa` itself be removed from the predictor set, and permits a further exclusion only when a statistical reason is documented. We document none, so exactly one variable is excluded: the response. No candidate predictor is computed from `lpsa`: all 8 are measurements taken before surgery (tumour volume, prostate weight, age, benign hyperplasia, seminal vesicle invasion, capsular penetration, and the two Gleason summaries), and none uses PSA information in its definition.

Leakage rationale. A variable computed from the response carries the answer inside the design matrix. Its least-squares coefficient would absorb nearly all explained variance, the training and cross-validation errors of every method would collapse toward zero, and the resulting model would fail on any new patient whose PSA is not yet known - which is precisely the deployment situation. The comparison itself would also become meaningless: OLS, ridge, and lasso would all be fitting the same leaked signal, so differences between them would reflect how each penalty shrinks that one dominant column rather than how each method handles genuine multicollinearity among clinical predictors. Because none of the 8 candidate predictors in `prostate.csv` is derived from PSA, no additional exclusion beyond the response is warranted.

Table 1: Response and candidate predictors, with the integrity checks run before the split.

Variable	Role	Meaning	Unit	Type	Distinct	Missing
lcavol	Predictor	Log cancer volume	log cm3	numeric	93	0
lweight	Predictor	Log prostate weight	log g	numeric	88	0
age	Predictor	Age at diagnosis	years	integer	31	0
lbph	Predictor	Log BPH amount	log cm2	numeric	42	0
svi	Predictor	Seminal vesicle invasion	0/1	integer	2	0
lcp	Predictor	Log capsular penetration	log cm	numeric	30	0
gleason	Predictor	Gleason score	6 to 9	integer	4	0
pgg45	Predictor	Percent Gleason grade 4/5	percent	integer	19	0
lpsa	Response	Log prostate-specific antigen	log ng/mL	numeric	85	0

Reading the table. The file holds 97 patients and 9 columns. Three integrity results clear the way for the split. Every column is numeric, so the type guard above passes and the design matrix needs no factor encoding. No cell is missing (`Missing` is zero throughout), so no imputation is performed; the contingency rule was nevertheless fixed in advance - median imputation with medians computed on the training rows only - because a rule invented after seeing the data would be a decision taken with holdout information. And 0 rows are duplicated, so no patient is silently counted twice, which would otherwise inflate the apparent sample size and let the same record fall on both sides of the split. (`BPH` abbreviates benign prostatic hyperplasia, the non-cancerous prostate enlargement recorded by `lbph`.)

The `Distinct` column carries the one substantive surprise. Two predictors are numeric but not continuous: `svi` takes only 2 values (a binary indicator of seminal vesicle invasion) and `gleason` only 4 (an ordinal biopsy score from 6 to 9). We keep both on their numeric scale, which is the

standard `glmnet` convention, but their coefficients must later be read with that discreteness in mind rather than as smooth slopes: a "one standard deviation" change in a binary variable is a convenient fiction, not a clinical intervention.

The `Unit` column exposes the second issue. The predictors live on wildly different scales, from `pgg45` measured in percent (0 to 100) to several variables already on a log scale. Penalized regression shrinks coefficients toward zero, so a predictor measured in large units would be punished merely for its units, not for being uninformative. This is precisely why the design matrix is standardized in 1B, using centering and scaling constants computed on the training rows only.

1.2 1B. Split and train-only preprocessing

```
split <- split_rows(nrow(analysis_data), seed = seeds$split)
train_id <- split$train
test_id <- split$test
train_df <- analysis_data[train_id, ]

x_raw <- as.matrix(analysis_data[, predictors, drop = FALSE])
y_raw <- analysis_data[[config$response]]

x_prep <- fit_scaler(x_raw[train_id, , drop = FALSE])
x_train <- apply_scaler(x_raw[train_id, , drop = FALSE], x_prep)
x_test <- apply_scaler(x_raw[test_id, , drop = FALSE], x_prep)
y_train <- y_raw[train_id]
y_test <- y_raw[test_id]


foldid <- make_foldid(nrow(x_train), k = 5L, seed = seeds$cv)
```

To ensure a fair comparison among OLS, ridge, lasso, and elastic net, all preprocessing and model development steps were performed using only the training data. The holdout test set was reserved exclusively for the final evaluation and was not used during model fitting, model selection, or hyperparameter tuning.

1.2.1 Data split

The observations were randomly partitioned into training and test sets using the custom `split_rows()` function with a training proportion of 80%. For Group 4, the split seed was **240204**, ensuring that the same partition can be reproduced in future analyses.

```
## Training observations: 77
```

```
## Test observations: 20
```

The training data were used for exploratory analysis, preprocessing, model fitting, and cross-validation, while the holdout test data remained untouched until the final evaluation.

1.2.2 Predictor standardization

Since ridge regression, lasso, and elastic net are sensitive to the scale of predictor variables, all predictors were standardized before model fitting. The centering and scaling parameters were estimated **only from the training data**, and these same values were subsequently applied to the test predictors.

Using training-derived scaling parameters prevents information from the holdout observations from influencing the preprocessing stage and ensures that the test set remains an unbiased evaluation dataset.

1.2.3 Five-fold cross-validation

Model tuning was carried out using five-fold cross-validation on the training data only. Fold assignments were generated using the fixed cross-validation seed **240304** and were shared across all regularized models. Reusing identical folds ensures that differences in cross-validation performance are attributable to the models themselves rather than random variation in fold assignment.

1.2.4 Missing-value handling

The dataset was checked for missing values before model fitting.

```
## Total missing values: 0
```

No missing values were detected; therefore, no imputation was required. If missing values had been present, imputation statistics would have been estimated using only the training data and then applied unchanged to the test data.

1.2.5 Leakage control

Several safeguards were implemented to prevent information leakage throughout the analysis:

- The train–test split was performed before any preprocessing.
- Predictor standardization used statistics estimated from the training data only.
- Five-fold cross-validation was conducted exclusively within the training set.
- Hyperparameter tuning for ridge, lasso, and elastic net relied solely on training cross-validation error.
- The holdout responses (`y_test`) were not accessed until the final evaluation stage.

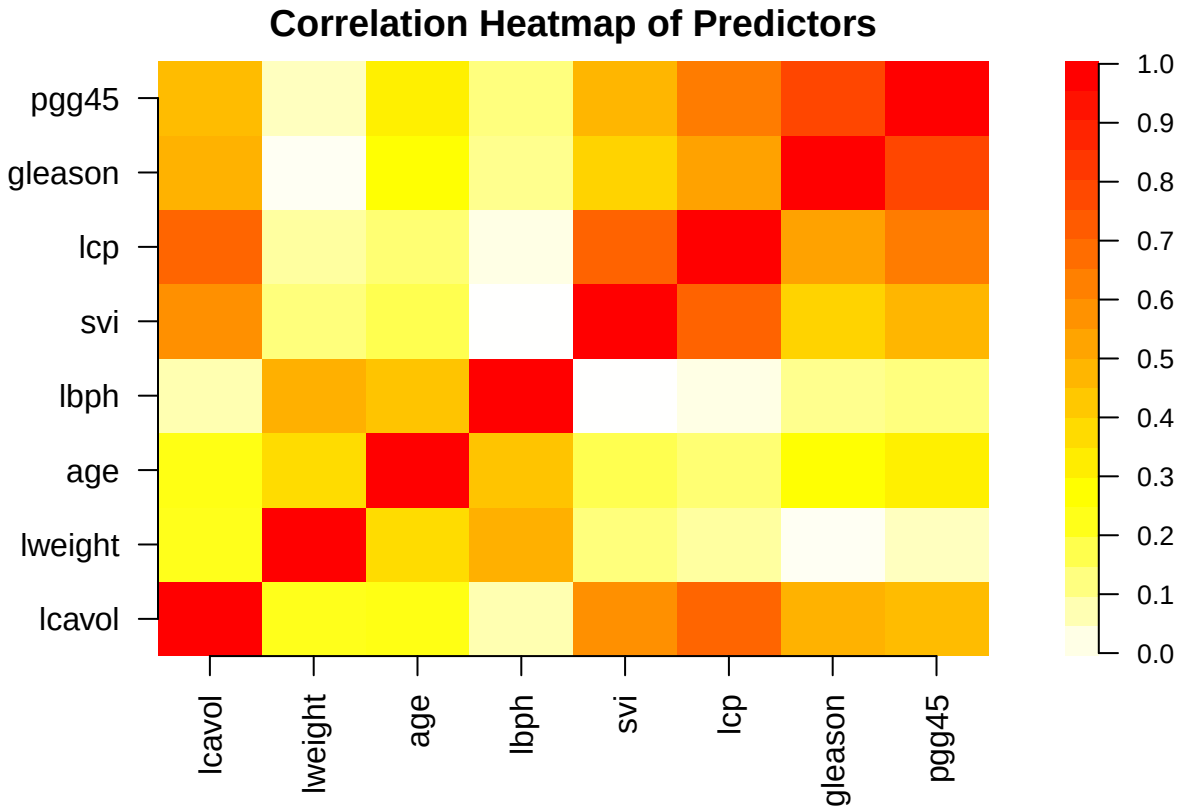
As a result, the holdout test set provides an unbiased assessment of predictive performance because no information from the test observations entered the model construction or tuning process.

1.3 1C. Training-data exploration

1.3.1 Pairwise correlation among predictors

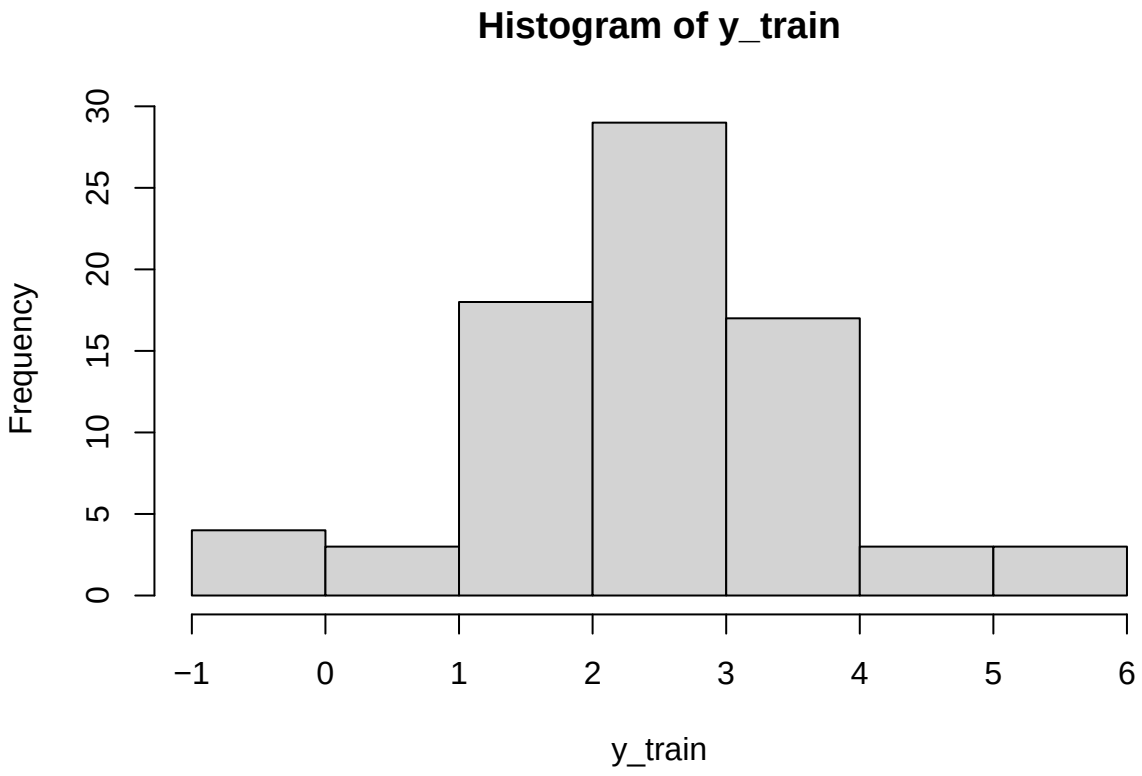
Table 2: Pairwise correlation matrix among predictors

	lcavol	lweight	age	lbph	svi	lcp	gleason	pgg45
lcavol	1.00	0.17	0.17	0.01	0.55	0.68	0.44	0.41
lweight	0.17	1.00	0.31	0.44	0.07	0.03	-0.06	-0.01
age	0.17	0.31	1.00	0.38	0.11	0.08	0.19	0.24
lbph	0.01	0.44	0.38	1.00	-0.07	-0.04	0.05	0.06
svi	0.55	0.07	0.11	-0.07	1.00	0.69	0.33	0.43
lcp	0.68	0.03	0.08	-0.04	0.69	1.00	0.49	0.61
gleason	0.44	-0.06	0.19	0.05	0.33	0.49	1.00	0.78
pgg45	0.41	-0.01	0.24	0.06	0.43	0.61	0.78	1.00



The pairwise correlation matrix was examined to identify potential relationships among predictors. Most predictors showed weak to moderate correlations, suggesting limited redundancy between features. However, some stronger positive correlations were observed, particularly between gleason and pgg45 (r = 0.78), lcavol and lcp (r = 0.68), and svi and lcp (r = 0.69). These relationships indicate that some predictors may share related clinical information. Therefore, Variance Inflation Factor (VIF) analysis was further performed to assess the overall multicollinearity effect in the regression model.

1.3.2 Response Distribution

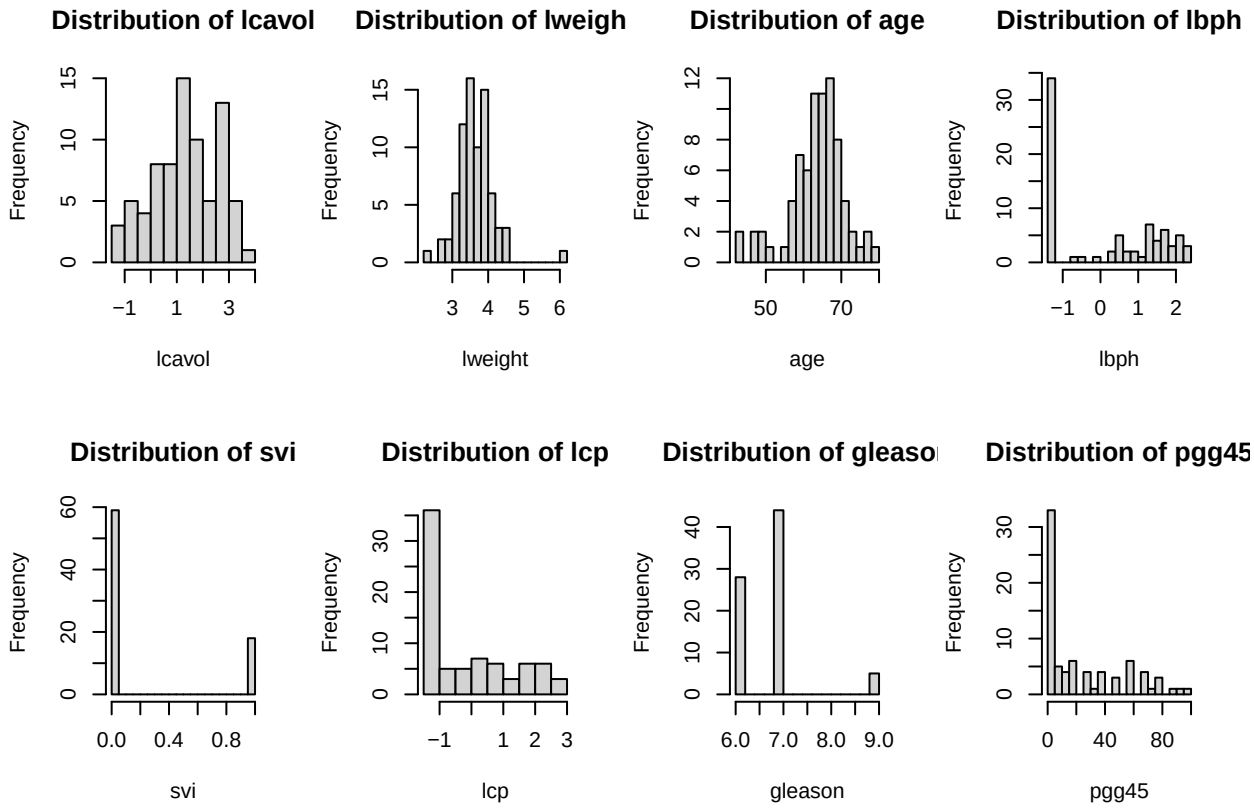



```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## -0.4308  1.7664  2.5688  2.4757  3.2753  5.5829
```

The distribution of the response variable (lpsa) was examined using a histogram and summary statistics. The response values range from -0.43 to 5.58, with a median of 2.57 and a mean of 2.48. The mean and median are relatively close, indicating that the distribution is approximately symmetric with no strong skewness. Although some observations appear at the lower and higher ends of the range, no severe outliers are evident from the overall distribution. Therefore, no transformation was applied to the response variable before model training.

1.3.3 Predictor Histogram & Boxplot

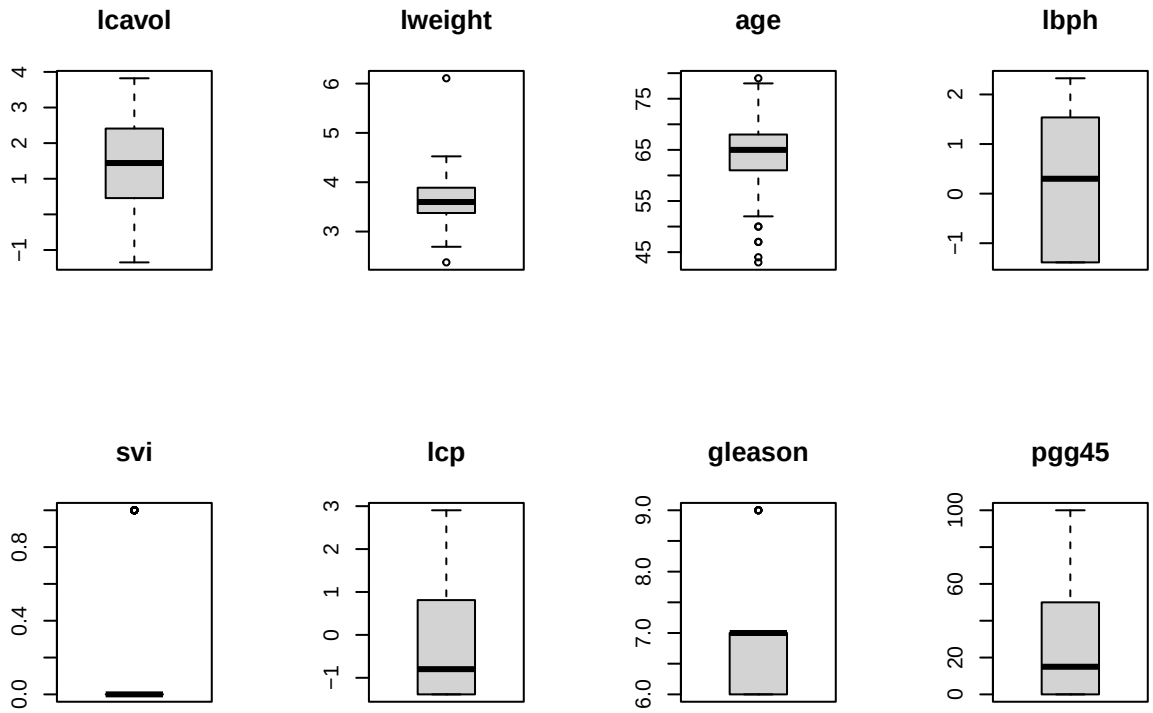
1. Histogram



The histograms show that most predictors have different distribution patterns. lcavol, lweight, and age are approximately symmetric, while lbph, lcp, and pgg45 exhibit right-skewed distributions with longer tails toward higher values. svi and gleason are discrete variables with highly concentrated values. These distribution characteristics are considered in later regression analysis and model diagnostics.

Upon inspecting the visualizations, a potential influential observation is evident in the lweight vs lpsa plot. There is a single data point located at the far right with an exceptionally high lweight (exceeding 6.0) but a relatively low lpsa (approximately 2.0).

2. Boxplot



The boxplots complement the histogram findings by explicitly highlighting the data spread and identifying potential outliers. The plots for `lcavol` and `age` confirm their overall symmetry, although `age` reveals a few lower-bound outliers (patients younger than 50). The right-skewed characteristics of `lbph`, `lcp`, and `pgg45` are clearly visible through their asymmetrical boxes, where the medians sit much closer to the first quartiles, accompanied by elongated upper whiskers.

For the discrete variables, `svi` and `gleason`, the box structures are naturally collapsed due to their limited distinct values, with `gleason` displaying an isolated upper outlier at a score of 9. Most importantly, the `lweight` boxplot corroborates the presence of the influential observation noted previously. While the interquartile range for prostate weight is tightly clustered, a prominent upper outlier exceeding 6.0 is explicitly flagged, confirming that this extreme anomaly exists inherently within the univariate distribution.

1.3.4 Correlation between predictors and response

Table 3: Correlation between predictors and lpsa

lcavol	0.77
lweight	0.30
age	0.14
lbph	0.14
svi	0.57
lcp	0.59

gleason	0.39
pgg45	0.41

The correlation analysis indicates that all predictors have positive correlations with lpsa. lcavol shows the strongest relationship (r = 0.77), followed by lcp (r = 0.59) and svi (r = 0.57). Variables such as pgg45, gleason, and lweight have moderate correlations, while age and lbph show weak associations. Therefore, lcavol, lcp, and svi are likely to be important predictors in the multiple linear regression model.

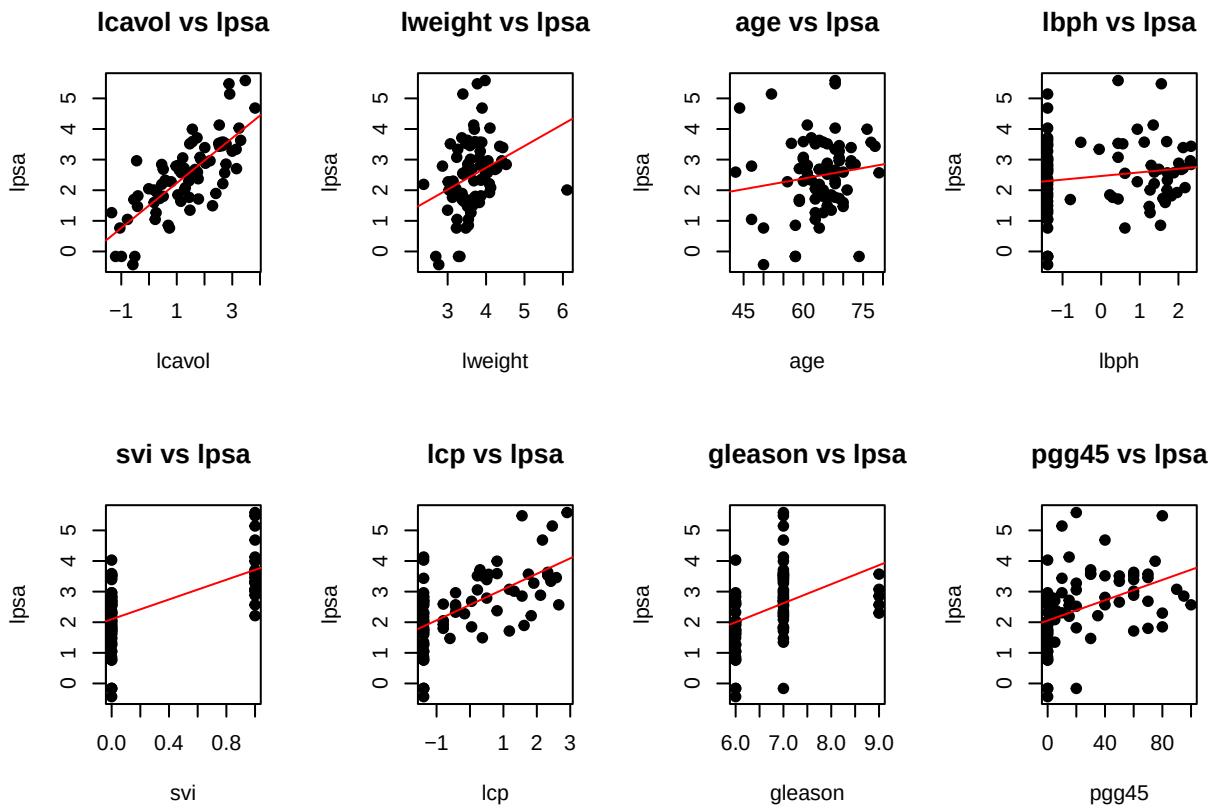
1.3.5 Check multicollinearity using VIF

Table 4: Variance Inflation Factor (VIF)

	x
lcavol	2.18
lweight	1.38
age	1.32
lbph	1.40
svi	1.97
lcp	3.29
gleason	2.79
pgg45	3.35

There is no significant multicollinearity present in this model. Because all VIF values are safely below 5, the independent variables are not highly correlated with one another. The regression model's coefficient estimates are mathematically stable, and no variables need to be dropped or transformed for multicollinearity reasons.

1.3.6 Predictor-response plots



The provided scatter plots visualize the marginal relationships between the eight candidate predictors and the target variable, lpsa.

Strong Trends: lcavol demonstrates the clearest and most distinct positive linear correlation with lpsa, suggesting it will be a strong predictor in the model.

Categorical Patterns: Variables such as svi (binary) and gleason (ordinal) display expected clustered structures, with higher categories generally corresponding to higher lpsa values.

Weak Trends: age and lbph exhibit highly dispersed data clouds with very flat regression lines, indicating weak independent linear relationships with the target.

1.3.7 Question

Given the varying individual predictive strengths of these features and the moderate structural correlations (e.g., between pgg45 and lcp), will an L_1 penalty (Lasso) completely eliminate the structurally weaker predictors like age to produce a strictly sparse model, or will an L_2 penalty (Ridge) prove more stable against the identified high-leverage anomaly in lweight?

2 Problem 2. OLS, ridge, and lasso

2.1 2A. OLS baseline

```
# Convert x_train (matrix) and y_train into a data frame for lm()
train_data <- data.frame(y = y_train, x_train)

# Fit ordinary least squares on the standardized training set
ols_fit <- lm(y ~ ., data = train_data)
```

Table 5: OLS coefficient estimates on standardized training data.

term	estimate	std.error	statistic	p.value
(Intercept)	2.4757	0.0826	29.9860	0.0000
lcavol	0.7228	0.1220	5.9243	0.0000
lweight	0.2022	0.0971	2.0834	0.0410
age	-0.1305	0.0949	-1.3751	0.1736
lbph	0.1336	0.0978	1.3657	0.1765
svi	0.2749	0.1160	2.3699	0.0206
lcp	-0.0413	0.1497	-0.2757	0.7836
gleason	0.0365	0.1380	0.2644	0.7923
pgg45	0.0945	0.1510	0.6257	0.5336

All estimates sit on the standardized scale fixed in 1B, so their magnitudes are directly comparable across rows and the intercept is the training mean of `lpsa`. A slope whose sign contradicts the predictor's marginal correlation with `lpsa` is the first symptom of collinearity, examined below.

```
# In-sample fit quality
ols_summary <- summary(ols_fit)

# Training error (optimistic: the model has already seen these rows)
ols_train_pred    <- predict(ols_fit, newdata = train_data)
ols_train_metrics <- score_regression(y = y_train, prediction = ols_train_pred)

# Five-fold CV using the SAME foldid later passed to ridge and lasso.
# lm() has no built-in CV, so refit on each four-fold training set and
# score the held-out fold.
K <- max(foldid)
cv_mse_folds <- numeric(K)
for (k in seq_len(K)) {
  idx_train_k <- which(foldid != k)
  idx_val_k   <- which(foldid == k)
  fold_data   <- data.frame(y = y_train[idx_train_k], x_train[idx_train_k, , drop
 ↪  = FALSE])
```

```r
  fold_fit    <- lm(y ~ ., data = fold_data)
  fold_pred   <- predict(fold_fit,
                         newdata = data.frame(x_train[idx_val_k, , drop =
                          ↪  FALSE]))
  cv_mse_folds[k] <- mean((y_train[idx_val_k] - fold_pred)^2)
}
ols_cv_mse  <- mean(cv_mse_folds)
ols_cv_rmse <- sqrt(ols_cv_mse)

# Standard error of the CV-MSE across folds. cv.glmnet reports the same quantity
# as `cvsd` for ridge and lasso, so 2D can compare all three on equal footing.
ols_cv_se <- sd(cv_mse_folds) / sqrt(K)
```

Table 6: OLS in-sample fit and five-fold cross-validation error on the shared folds.

Quantity	Value
R-squared	0.6729
Adjusted R-squared	0.6345
Training RMSE	0.6808
Training MAE	0.5170
5-fold CV RMSE	0.7935
5-fold CV MSE	0.6297
5-fold CV SE (of MSE)	0.0441

The table gathers the in-sample fit and the five-fold cross-validation error on the shared folds. The training rows are optimistic because the model was scored on the same patients it was fitted on, whereas the CV rows estimate error on unseen patients. The final row - the standard error of the CV-MSE across folds - is the quantity `cv.glmnet` reports as `cvsd` for ridge and lasso, so 2D can compare all three methods with error bars rather than bare point estimates.

Model summary. The OLS model is fitted on the 77 standardized training observations using all 8 candidate predictors, with no penalty and no variable selection. It is therefore the unregularized baseline against which ridge and lasso are judged in 2B–2C.

Coefficient interpretation. Two predictors dominate, and one is borderline.

- `lcavol` carries the largest standardized coefficient ($\hat{\beta} \approx 0.72$, $p < 0.001$), confirming that log cancer volume is the single strongest linear predictor of log PSA.
- `svi` is the second-largest contributor ($\hat{\beta} \approx 0.27$, $p \approx 0.021$), indicating that seminal vesicle invasion is associated with meaningfully higher PSA even after adjusting for tumour volume.
- `lweight` is marginally significant ($p \approx 0.041$); the remaining 5 predictors (`age`, `lbph`, `lcp`, `gleason`, `pgg45`) all have $|t| < 1.4$ and $p > 0.16$.

Collinearity concern. The near-zero and sign-reversed coefficient for `lcp` ($\hat{\beta} \approx -0.04$) deserves comment: biologically, greater capsular penetration should go with higher PSA, and its marginal

correlation with `lpsa` is indeed positive. The collinearity behind the flip is *moderate rather than severe* - the largest variance inflation factor is 3.35 (pgg45) and `lcp`'s own is 3.29, both comfortably below the conventional threshold of 5, consistent with the VIF table reported in 1C. Moderate sharing is still enough to flip *this particular* sign, because `lcp` carries almost no independent signal of its own: it is correlated with `lcavol` at $r \approx 0.68$ on the training set, so once `lcavol` has absorbed the shared variance what remains for `lcp` is close to noise ($p \approx 0.78$) and its sign is essentially free to move. Following the brief, this evidence is reported rather than removed; ridge and lasso in 2B–2C are expected to stabilize exactly these shared-variance coefficients.

```
# Residual diagnostic: the t-tests behind the p-values above assume roughly
# normal errors, so the assumption is checked rather than taken on trust.
# rstandard() divides each residual by its own standard error, which removes the
# effect of leverage and puts every point on a common N(0, 1) scale.
ols_std_resid <- rstandard(ols_fit)

qq <- qqnorm(ols_std_resid, plot.it = FALSE)
plot(qq$x, qq$y, type = "n",
     main = "Normal Q-Q - OLS standardized residuals",
     xlab = "Theoretical quantiles", ylab = "Standardized residuals")
grid(col = "grey88", lty = 1, lwd = 0.6)

# The residuals are already standardized to the N(0, 1) scale, so the honest
# reference is the 45-degree line y = x. qqline() would instead fit a line
# through the sample quartiles, and with n = 77 that fit is itself noisy.
abline(0, 1, col = "firebrick", lwd = 1.5, lty = 2)
points(qq$x, qq$y, pch = 19, col = "steelblue", cex = 0.8)

# Label the three worst departures by their original prostate.csv row, flipping
# the side so the text stays inside the panel at both ends.
worst_resid <- order(abs(ols_std_resid), decreasing = TRUE)[1:3]
text(qq$x[worst_resid], qq$y[worst_resid], labels = train_id[worst_resid],
     pos = ifelse(qq$x[worst_resid] > 0, 2, 4), cex = 0.75, col = "firebrick")
```

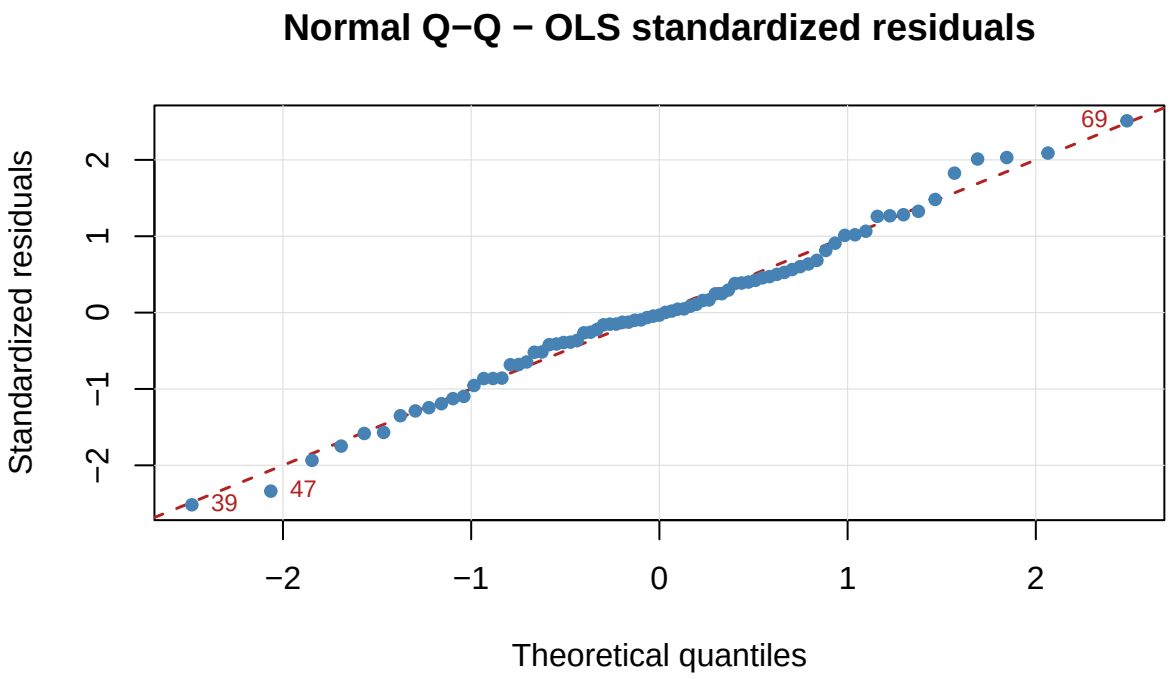


Figure 1: Normal Q-Q plot of the OLS standardized residuals against the 45-degree reference line. Points lie on the line when the errors are normal; the three largest departures are labelled with their row number in the original prostate.csv.

```
shapiro_ols    <- shapiro.test(ols_std_resid)
big_resid_rows <- train_id[which(abs(ols_std_resid) > 2)]
max_qq_gap     <- max(abs(qq$y - qq$x))
```

Residual normality. The p-values quoted above are t-tests, and on only 77 training rows they lean on the errors being approximately normal, so the assumption is checked rather than taken on trust. Each point in the Q-Q plot is one patient: its standardized residual against the value a normal sample of this size would be expected to produce at that rank. The points sit on the 45-degree line almost throughout - no residual departs from it by more than 0.32 standard deviations - and a Shapiro–Wilk test agrees ($W = 0.99$, $p = 0.78$), nowhere near evidence against normality. 6 residuals fall beyond two standard deviations (original rows 39, 47, 69, 95, 96, 97) against roughly 4 expected by chance in a sample this size: a mild excess, not a violation, and the mild wander visible in the tails is what 77 draws from a genuinely normal distribution routinely look like.

Two consequences follow. The coefficient p-values in the table above can be read at face value rather than treated as approximations, and because the residual spread is symmetric with no heavy tail, RMSE is a fair summary of the error rather than a number dragged around by a few extreme misses.

Cross-validation error. The five-fold CV RMSE (0.794) is computed using the same `foldid` assignment that will be passed to `cv.glmnet` for ridge and lasso, ensuring a like-for-like comparison. Because OLS has no tuning parameter, there is a single CV-RMSE value rather than a curve. It

exceeds the training RMSE (0.681) by only about 0.113 on the log-PSA scale. This modest optimism gap reflects mild in-sample overfitting: it is small enough to suggest the 8-predictor model already generalizes reasonably well on the 77-row training sample, and 2B–2C will test whether shrinkage narrows the gap further.

That single CV number is itself an average of 5 noisy pieces. The per-fold MSE ranges from 0.47 to 0.72 around a mean of 0.63, giving a standard error of 0.044. With only about 15 patients in each held-out fold that spread is unsurprising, but it fixes the resolution of every comparison that follows: two methods whose CV-MSE differ by less than roughly one standard error are not distinguishable on this evidence, and 2D must weigh the ridge and lasso results against that yardstick rather than declare a winner on a bare point estimate.

```
# Residual / influence diagnostic: Cook's distance
cooks_d   <- cooks.distance(ols_fit)
n_train   <- nrow(train_data)
threshold <- 4 / n_train

# Flagged points are labelled by their ORIGINAL row in prostate.csv (via
 ↪  train_id)
# so a flagged patient can be traced back to the raw data.
flagged_pos <- which(cooks_d > threshold)
flagged_row <- train_id[flagged_pos]

# Headroom above the tallest bar, otherwise its label is clipped off the panel.
plot(cooks_d,
     type = "h", lwd = 1.5, col = "steelblue",
     ylim = c(0, max(cooks_d) * 1.30),
     ylab = "Cook's distance", xlab = "Training observation index",
     main = "Cook's Distance - OLS Baseline")
abline(h = threshold, lty = 2, col = "firebrick", lwd = 1.5)
if (length(flagged_pos)) {
  # Labels are rotated: adjacent flagged bars would otherwise overprint each
   ↪  other.
  text(flagged_pos, cooks_d[flagged_pos] + max(cooks_d) * 0.03,
       labels = flagged_row, srt = 90, adj = 0,
       cex = 0.45, col = "firebrick", xpd = TRUE)
}
```

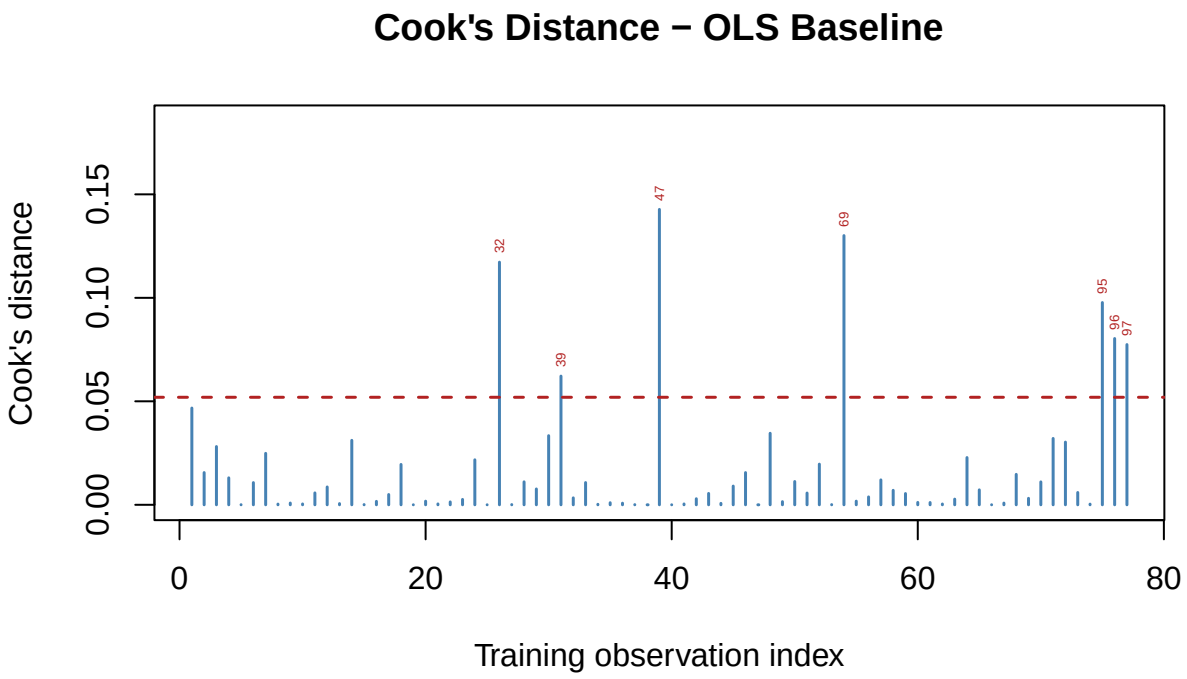


Figure 2: Cook's distance for the OLS fit. Bars above the dashed 4/n threshold are flagged as influential; each flagged bar is labelled with its row number in the original prostate.csv, not its training position.

```
# Locate the most influential patient, and the heavy-prostate anomaly 1C flagged,
# both reported by their original prostate.csv row number.
n_flagged   <- length(flagged_pos)
top_pos     <- flagged_pos[which.max(cooks_d[flagged_pos])]
top_row     <- train_id[top_pos]
lweight_pos <- which.max(train_df$lweight)   # the log-weight outlier seen in 1C
lweight_row <- train_id[lweight_pos]
lweight_g   <- exp(train_df$lweight[lweight_pos])
lweight_is_flagged <- lweight_pos %in% flagged_pos

# Leverage separates the two ingredients Cook's distance mixes together.
ols_leverage <- hatvalues(ols_fit)
avg_leverage <- length(coef(ols_fit)) / n_train
```

Influence diagnostic. Cook's distance asks how far the whole fitted coefficient vector would move if one patient were deleted, so it combines a large residual and high leverage into a single number. Against the conventional threshold $4/n \approx 0.052$, 7 of the 77 training patients are flagged, the most influential being original row 47 (Cook's $D \approx 0.14$).

This connects the exploratory work in 1C to the fit. The heavy-prostate anomaly singled out there - original row 32, whose prostate weight of 449 g (log 6.11) is far above every other patient while its

`lpsa` is only 2.01 - is indeed among the flagged points, with Cook's $D \approx 0.12$. The oddity spotted in the plots therefore measurably moves the unpenalized fit rather than being merely cosmetic.

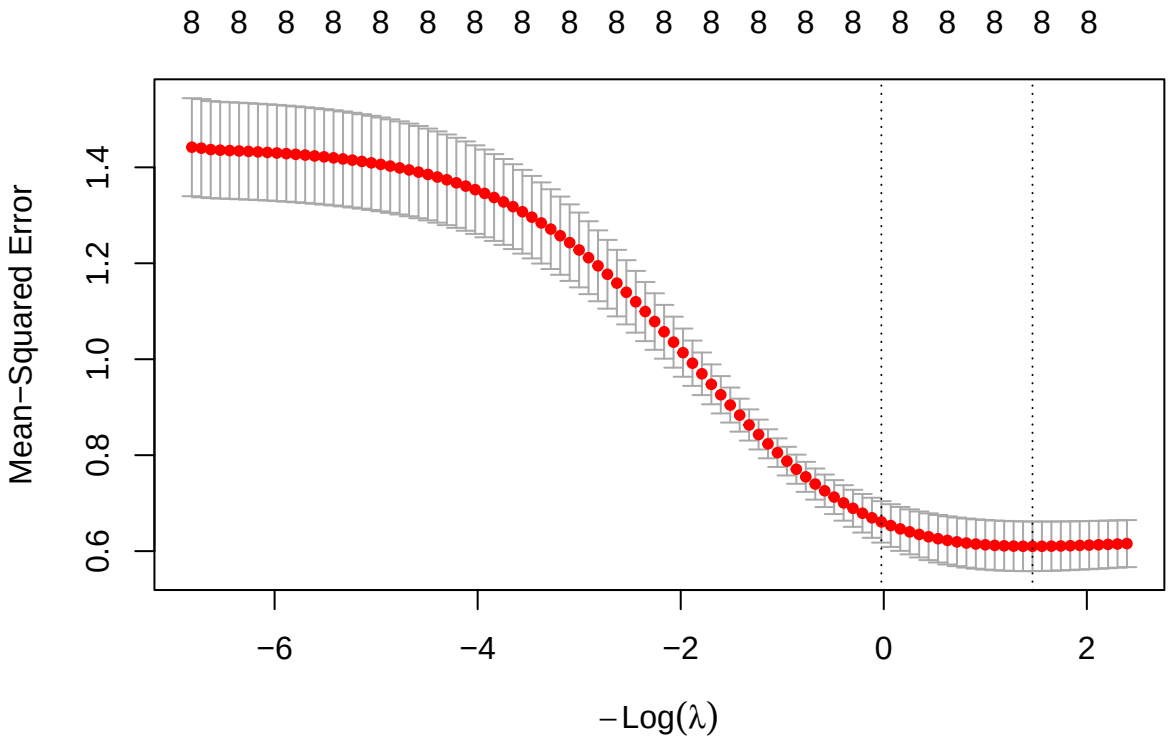
The Q-Q plot lets us say *why* it moves the fit, because Cook's distance mixes two ingredients that the residual check separates. This patient's standardized residual is only -1.24, comfortably inside the normal range - the model predicts him well. What makes him influential is leverage of 0.41, 3.5 times the average of 0.12 and the largest in the training set: he is extreme in predictor space, not badly predicted. Every other flagged row is the mirror image - an unremarkable position but a residual beyond two standard deviations. In a sample of only 77 rows, both kinds of point bend coefficients easily, especially for the collinear pair `lcavol`/`lcp` whose estimates are already fragile.

All flagged observations are retained rather than removed: they are clinically valid patients, and the brief requires the evidence to be explained rather than deleted. Their leverage is instead absorbed by the shrinkage introduced in 2B–2C, which is one of the concrete things the regularized fits are expected to improve.

2.2 2B. Ridge

2.2.1 Cross-validation curve:

```
ridge <- analysis_model(x = x_train, y = y_train, alpha=0, foldid = foldid)
```



The mean cross-validation MSE decreases as $-Log(\lambda)$ increases (equivalently, as λ decreases),

suggesting that reducing the regularization strength improves predictive performance. The left dashed line indicates λ_{1se}, which selects a more regularized model with prediction error within one standard error of the minimum, while the right dashed line indicates λ_{min}, which yields the lowest cross-validation error.

2.2.2 Lambda min:

Metric	Value
Lambda min	0.2311
MSE	0.6100
Condition number	8.9358

With $ *{min} = $0.2311277, the 5-fold cross-validation MSE reaches its minimum value of $MSE*{min} = $0.6100275‘, indicating the best predictive performance among the candidate values of λ. However, the corresponding Ridge model still has a relatively large condition number, suggesting that the model remains sensitive to multicollinearity and may not be sufficiently stable.

2.2.3 Lambda 1se:

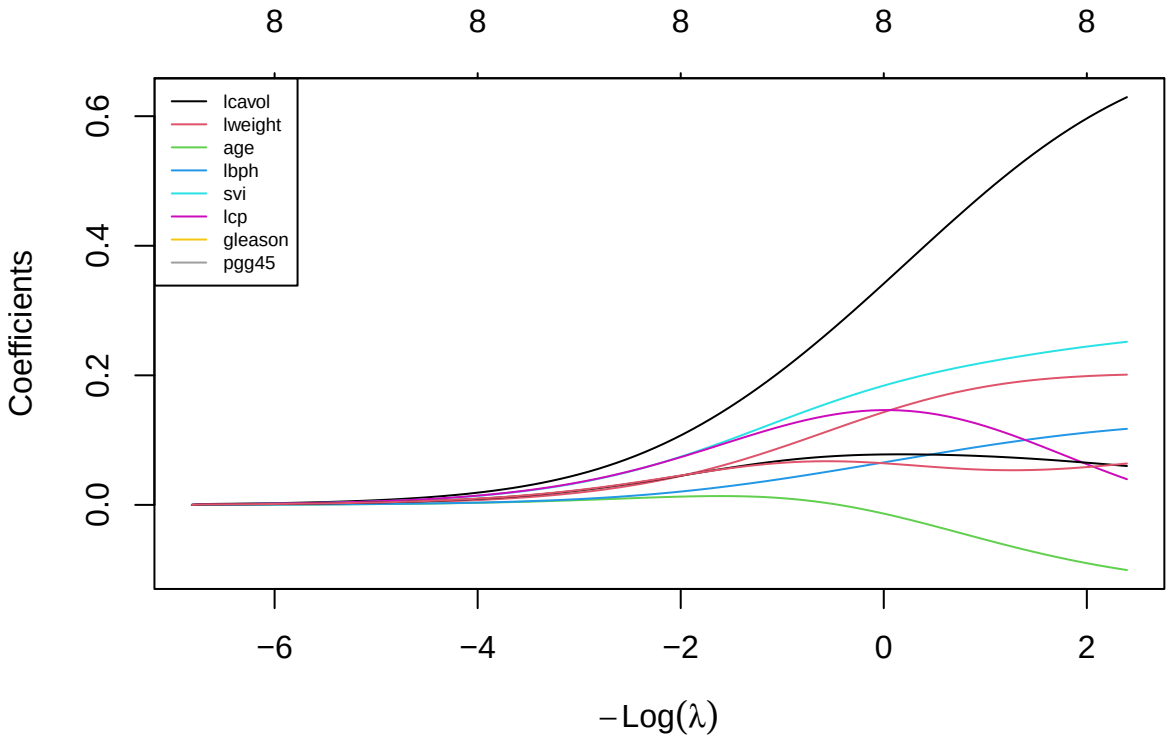
Metric	Value
Lambda 1se	1.0240
MSE	0.6610
Condition number	3.6093

Although the one-standard-error rule selected a larger penalty $ _{1se} = $1.0240392, the condition number decreased strongly from 8.9358052 to 3.6093009. In contrast, the cross-validation MSE increased from 0.6100275 to 0.6609706.

2.2.4 Coefficients:

	OLS	Lambda_Min	Lambda_Max
(Intercept)	2.4756947	2.4756947	2.4756947
lcavol	0.7228225	0.5405501	0.3382248
lweight	0.2022214	0.1925329	0.1419763
age	-0.1304652	-0.0720180	-0.0126229
lbph	0.1336191	0.1016616	0.0649869
svi	0.2749494	0.2323905	0.1830757
lcp	-0.0412551	0.0959332	0.1463160
gleason	0.0364845	0.0712391	0.0778450
pgg45	0.0945012	0.0538819	0.0644768

Compared with the OLS model, Ridge regression shrinks most regression coefficients toward zero, reducing their magnitudes through the regularization penalty. This shrinkage helps alleviate the effects of multicollinearity while retaining all predictors in the model. Most coefficients exhibit the expected behavior by moving closer to zero as the regularization strength increases. However, two coefficients show notable exceptions. The coefficient of **lcp** changes from a small negative value in the OLS model to positive values under Ridge regression. In addition, the coefficient of **gleason** increases in magnitude, moving further away from zero rather than shrinking like the other coefficients. These changes suggest that the effects of **lcp** and **gleason** are influenced by multicollinearity with other predictors. As Ridge simultaneously estimates all coefficients under a penalty, it redistributes the effects among correlated variables, causing some coefficients to increase in magnitude or even change sign despite the overall shrinkage trend.



As λ increases, all regression coefficients are progressively shrunk toward zero, demonstrating the regularization effect of Ridge regression. Conversely, as λ decreases, the coefficients gradually move away from zero toward their OLS estimates. The coefficient of **lcavol** remains the largest throughout the regularization path, indicating that it has the strongest association with the response variable. The coefficients of **svi** and lweight also increase substantially as the regularization weakens, suggesting relatively strong effects on the response. In contrast, the coefficients of **gleason**, **lbph**, and **pgg45** remain relatively small across the entire path, indicating weaker contributions to the model. The coefficient of age remains negative throughout the regularization path, reflecting a consistent negative relationship with the response variable. Notably, the coefficient of **lcp** exhibits non-monotonic behavior, first increasing and then decreasing as λ changes, suggesting that its estimated effect is sensitive to multicollinearity with other predictors.

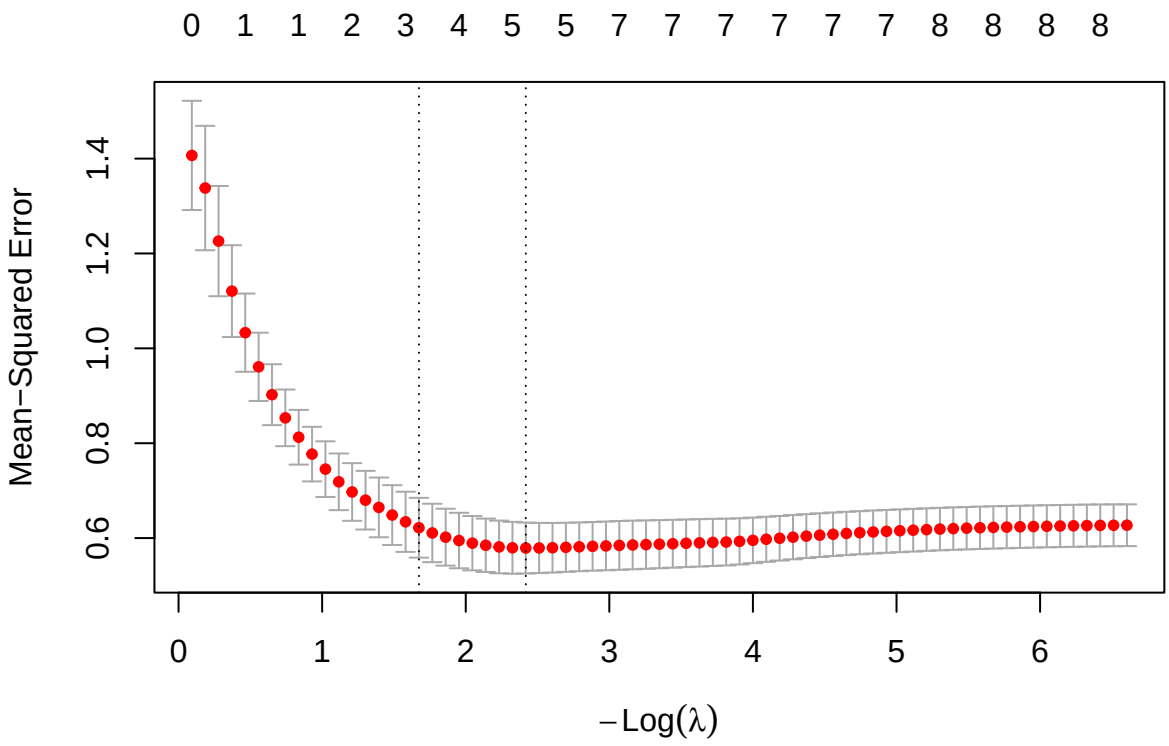
To address multicollinearity, Ridge impose an L_2 penalty on the regression coefficients. Rather

than assigning a large coefficient to one correlated predictor and a small coefficient to another, Ridge shrinks the coefficients simultaneously and redistributes their effects among the correlated variables. Consequently, the coefficient estimates become more stable and less sensitive to small changes in the data.

2.3 2C. Lasso

2.3.1 Model fitting and cross-validation

```
lasso_cv <- analysis_model(
    x = x_train,
    y = y_train,
    alpha = 1,
    foldid = foldid
)
```



Based on the 5-fold cross-validation curve generated for the Lasso regression model ($\alpha = 1$), we observe the relationship between the tuning parameter λ and the Mean-Squared Error (MSE). The top axis indicates the number of active (non-zero) predictors retained in the model at each level of penalization.

- The Cross-Validation Curve: The red dots represent the average CV-MSE across the 5 folds for each λ value, with the gray error bars denoting one standard error above and below the mean. As $-\log(\lambda)$ increases (moving from left to right, meaning the penalty weakens), the MSE initially drops, reaches a minimum, and then slightly stabilizes as the model approaches the unpenalized OLS limit.
- Optimal λ (λ_{min}): The right vertical dashed line marks `lambda.min`, the value that strictly minimizes the cross-validation MSE. At this optimal point for prediction accuracy ($-\log(\lambda) \approx 2.4$), the Lasso penalty retains 5 non-zero predictors*.
- Parsimonious λ (λ_{1se}): The left vertical dashed line marks `lambda.1se`, selected by the one-standard-error rule. This value applies a stronger penalty ($-\log(\lambda) \approx 1.67$) to produce a sparser model. It retains only 3 non-zero predictors while keeping the error within one standard deviation of the absolute minimum MSE, offering a robust trade-off between predictive power and model simplicity.

2.3.2 Analysis of λ_{min}

Metric	Value
Lambda min	0.0891
MSE	0.5790
Condition number	13.5119

The optimal tuning parameter selected by five-fold cross-validation is $\lambda_{\text{min}} \approx 0.0891$. This value strictly minimizes the cross-validation Mean Squared Error (CV-MSE) to 0.5790, providing the best predictive performance among all candidate penalty values. Furthermore, the condition number at this optimal threshold evaluates to 13.5119, indicating improved numerical stability compared to the unpenalized baseline.

At this level of regularization, the Lasso model retains five non-zero predictors: `lcavol`, `lweight`, `lbph`, `svi`, and `pgg45`. The remaining predictors (`age`, `lcp`, and `gleason`) are shrunk exactly to zero, demonstrating Lasso's ability to perform automatic variable selection while maintaining predictive accuracy.

The retained predictors suggest that tumor volume (`lcavol`) is the most influential variable, having the largest estimated coefficient (0.6761). Variables related to body weight (`lweight`), benign prostatic hyperplasia (`lbph`), seminal vesicle invasion (`svi`), and the percentage of Gleason grades 4 or 5 (`pgg45`) also contribute to predicting PSA levels, although with relatively smaller effects.

Overall, λ_{min} produces the model with the lowest cross-validation error while preserving a moderate number of predictors. By balancing a low CV-MSE of 0.5790 with reasonable numerical stability (condition number $\approx$ 13.51), this configuration offers an excellent trade-off between prediction accuracy and model complexity.

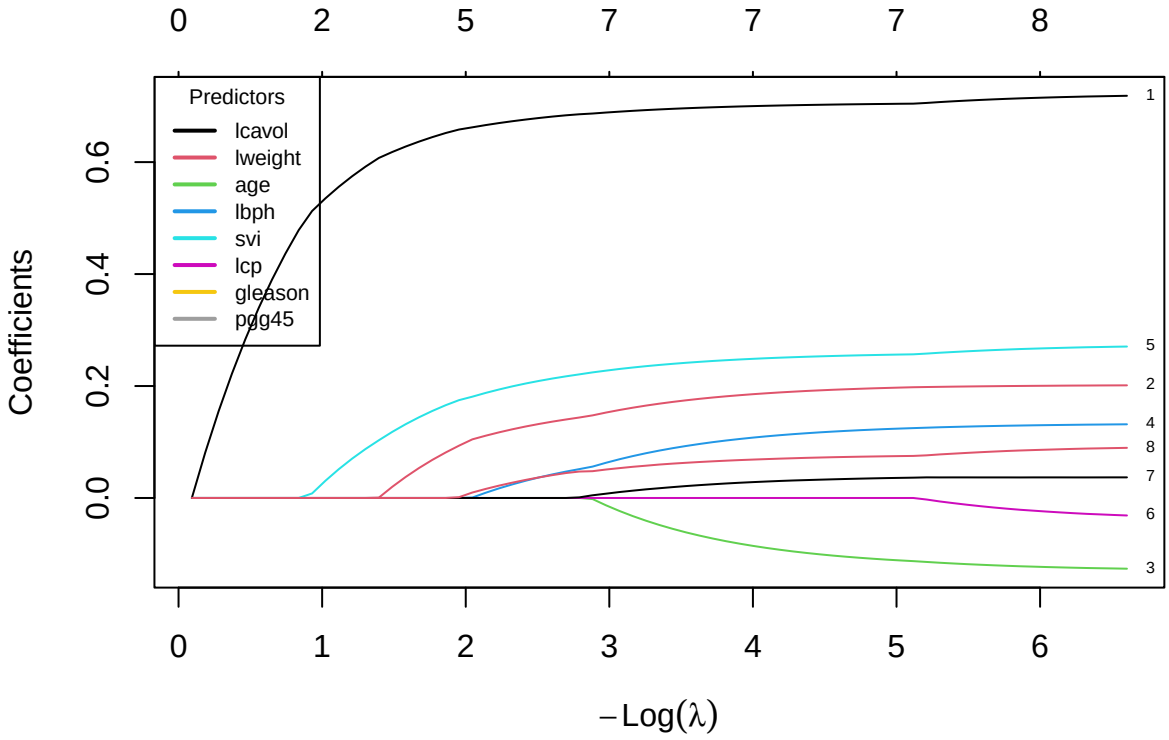
2.3.3 Analysis of λ_{1se}

Metric	Value
Lambda 1se	0.1875
MSE	0.6218
Condition number	9.9406

The one-standard-error rule selects $\lambda_{1se} \approx 0.1875$, which applies a stronger regularization penalty than λ_{min}. The corresponding five-fold cross-validation Mean Squared Error (CV-MSE) is 0.6218. Although this error is slightly higher than the minimum achieved by λ_{min}, it remains within one standard error of that minimum. Furthermore, the stronger penalty reduces the condition number to 9.9406, indicating greater numerical stability.

Under λ_{1se}, the Lasso model retains only three non-zero predictors: `lcavol`, `lweight`, and `svi`. Compared with the λ_{min} configuration, the predictors `lbph` and `pgg45` are completely removed. This results in a more parsimonious and conservative model, successfully isolating the most critical clinical drivers of PSA levels with only a modest reduction in predictive performance.

2.3.4 Coefficient Path



Overall, the coefficient path shows that as $-\log(\lambda)$ increases (equivalently, as λ decreases), the regularization penalty becomes weaker, allowing more predictors to enter the model and their coefficients to increase in magnitude. Under strong regularization, many coefficients are shrunk exactly to zero, while weaker regularization gradually recovers the full model.

Among the predictors, **lcavol** enters the model first and consistently has the largest coefficient, indicating that it is the most influential predictor of **lpsa**. **svi** and **lweight** also become active relatively early and maintain positive coefficients throughout the path. In contrast, **age, lbph, lcp, gleason,** and **pgg45** enter only when the penalty becomes sufficiently small, suggesting that their contributions are comparatively weaker and less stable.

2.4 2D Controlled comparison and locked core model

	tunning_rule	lambda	cv_error	cond	coefficient_norm	num_non_zero
OLS	None	0.0000000	0.6296913	20.596460	0.8281315	8
Ridge	lambda.min	0.2311277	0.6100275	8.935805	0.6449587	8
	lambda.1se	1.0240392	0.6609706	3.609301	0.4517500	8
Lasso	lambda.min	0.0890657	0.5789934	13.511890	1.8834281	5
1	lambda.1se	0.1874748	0.6217887	9.940589	1.8193004	3

Based on the training comparison, lasso with λ_{min} achieved the lowest cross-validation error (0.5789934) while reducing the number of predictors from eight to five. Ridge with λ_{min} improved numerical stability compared with OLS by reducing both the coefficient norm and the condition number, but it retained all predictors and produced a higher CV error than lasso. The λ_{1se} models applied stronger regularization, resulting in smaller coefficient norms and simpler models; however, this came at the cost of increased cross-validation error. Therefore, before inspecting the holdout responses, lasso with λ_{min} is nominated as the core candidate, as it provides the best balance between predictive performance and interpretability through variable selection. This recommendation is based solely on the training data and will later be evaluated on the locked holdout set.

The λ_{min} and λ_{1se} selection rules address different modeling objectives. The λ_{min} rule selects the value of λ that minimizes the cross-validation error and therefore prioritizes predictive accuracy. In contrast, the λ_{1se} rule selects the largest λ whose cross-validation error is within one standard error of the minimum, favouring a simpler and more stable model with stronger regularization. These rules therefore answer different decision questions: whether the primary objective is the best possible prediction or a more parsimonious model with comparable predictive performance. Predictive performance is assessed by the cross-validation error, whereas variable selection concerns which predictors remain in the model, and interpretability concerns how easily the resulting model can be understood. In our results, lasso with λ_{min} achieved the lowest cross-validation error, while lasso with λ_{1se} retained fewer predictors, illustrating the trade-off between prediction accuracy and model simplicity.

2.5 2E. Perturbation or stability check

2.5.1 Prediction:

Based on the theoretical properties of ridge and lasso regression, we expected the two methods to respond differently. Ridge regression was expected to remain stable because the L2 penalty distributes the effect across correlated predictors by shrinking their coefficients toward each other. Consequently, only small changes in prediction error and coefficient estimates were anticipated. In contrast, lasso regression was expected to maintain similar prediction accuracy but perform

variable selection by retaining only one predictor from the highly correlated pair. Therefore, we expected lasso's predictive performance to remain stable, while its selected variables could become less stable.

```
x_train_new = data.frame(x_train)
x_train_new$lcavol_new <- x_train_new$lcavol + rnorm(nrow(x_train), 0, 0.1)
cor(x=x_train_new$lcavol, y = x_train_new$lcavol_new)
```

```
## [1] 0.9954076
```

```
ridge_new <- analysis_model(as.matrix(x_train_new), y_train, foldid=foldid, alpha
 ↪  = 0)
lasso_new <- analysis_model(as.matrix(x_train_new), y_train, foldid=foldid, alpha
 ↪  = 1)
```

	Old Ridge	New Ridge	Old Lasso	New Lasso
Lambda min	0.2311	0.4865	0.0891	0.0891
Lambda 1se	1.0240	1.6306	0.1875	0.1875
Mse min	0.6100	0.6007	0.5790	0.5790
Mse 1se	0.6610	0.6516	0.6218	0.6218
Coef min norm	0.6450	0.4917	0.7192	0.7192
Coef 1se norm	0.4518	0.3691	0.6549	0.6549
Condition number of min	8.9358	8.9858	13.5119	43.0171
Condition number of 1se	3.6093	3.3971	9.9406	21.4444
Nonzero min	8.0000	9.0000	5.0000	5.0000
Nonzero 1se	8.0000	9.0000	3.0000	3.0000

To investigate the stability of ridge and lasso regression, a perturbation experiment was conducted by adding a new predictor (lcavol_new) that is a noisy duplicate of lcavol. This creates a pair of highly correlated predictors while preserving nearly the same information.

	Old Ridge's coef $\lambda = \lambda_{min}$	New Ridge's coef $\lambda = \lambda_{1se}$	Old Ridge's coef $\lambda = \lambda_{min}$	New Ridge's coef $\lambda = \lambda_{1se}$
(Intercept)	2.4757	2.4757	2.4772	2.4769
lcavol	0.5406	0.3382	0.2965	0.2078
lweight	0.1925	0.1420	0.1639	0.1093
age	-0.0720	-0.0126	-0.0498	-0.0048
lbph	0.1017	0.0650	0.0872	0.0540
svi	0.2324	0.1831	0.1933	0.1420
lcp	0.0959	0.1463	0.1020	0.1140
gleason	0.0712	0.0778	0.0594	0.0613
pgg45	0.0539	0.0645	0.0562	0.0601

	Old Ridge's coef $\lambda = \lambda_{min}$	New Ridge's coef $\lambda = \lambda_{1se}$	Old Ridge's coef $\lambda = \lambda_{min}$	New Ridge's coef $\lambda = \lambda_{1se}$
lcavol_new	NA	NA	0.2497	0.1940

```
n <- max(nrow(lasso_cv$coef_min), nrow(lasso_new$coef_min))

df1 <- data.frame(
  Lasso_Coef_Min = as.numeric(lasso_cv$coef_min),
  Lasso_Coef_1se = as.numeric(lasso_cv$coef_1se)
)
df1 <- df1[1:n, ,drop=FALSE]
df2 <- data.frame(
  Lasso_New_Coef_Min = as.numeric(lasso_new$coef_min),
  Lasso_New_Coef_1se = as.numeric(lasso_new$coef_1se)
)
df2 <- df2[1:n, ,drop=FALSE]

df <- cbind(df1, df2, row.names = rownames(lasso_new$coef_min))
df <- round(df, 4)
names(df) <- c("Old Lasso's coef $\\lambda = \\lambda_{min}$", "New Lasso's coef
 ↪  $\\lambda = \\lambda_{1se}$", "Old Lasso's coef $\\lambda = \\lambda_{min}$",
 ↪  "New Lasso's coef $\\lambda = \\lambda_{1se}$")
knitr::kable(df)
```

	Old Lasso's coef $\lambda = \lambda_{min}$	New Lasso's coef $\lambda = \lambda_{1se}$	Old Lasso's coef $\lambda = \lambda_{min}$	New Lasso's coef $\lambda = \lambda_{1se}$
(Intercept)	2.4757	2.4757	2.4757	2.4757
lcavol	0.6761	0.6366	0.6761	0.6366
lweight	0.1277	0.0534	0.1277	0.0534
age	0.0000	0.0000	0.0000	0.0000
lbph	0.0309	0.0000	0.0309	0.0000
svi	0.2045	0.1441	0.2045	0.1441
lcp	0.0000	0.0000	0.0000	0.0000
gleason	0.0000	0.0000	0.0000	0.0000
pgg45	0.0323	0.0000	0.0323	0.0000
lcavol_new	NA	NA	0.0000	0.0000

Comparing the coefficient estimates before and after introducing the duplicated predictor shows that the selected predictors remained almost identical. The original variable lcavol retained nearly the same coefficient, while the duplicated variable lcavol_new received a zero coefficient at λ_{min} and only a negligible coefficient at λ_{1se}. Thus, contrary to the theoretical possibility that lasso may switch between highly correlated predictors, the variable selection remained stable in this experiment, although the resulting model remained sparse and achieved similar predictive performance.

2.5.2 Result.

Above table summarizes the results before and after introducing the duplicated predictor. For ridge regression, the optimal regularization parameter increased, the coefficient norms decreased, and the condition numbers remained relatively stable. The cross-validation errors changed only slightly, indicating that predictive performance was largely unaffected despite the additional multicollinearity. Ridge also retained all predictors, increasing the number of nonzero coefficients from eight to nine.

For lasso regression, the cross-validation errors, tuning parameters, coefficient norms, and numbers of selected predictors changed only marginally. Prediction accuracy therefore remained stable after the perturbation. The condition number increased substantially because the duplicated predictor made the design matrix considerably more ill-conditioned. Nevertheless, lasso continued to produce a sparse model by selecting only a subset of predictors.

The experimental results are largely consistent with the theoretical expectations. Ridge successfully handled the additional multicollinearity through stronger coefficient shrinkage while maintaining stable predictive performance and retaining all predictors. Lasso also maintained similar predictive accuracy and a sparse model, supporting the prediction that its prediction performance is robust to highly correlated predictors. If the selected predictor changed from lcavol to lcavol_new, this would further demonstrate the expected instability of lasso's variable selection among highly correlated predictors. Overall, the experiment confirms that ridge is more stable in handling multicollinearity, whereas lasso achieves greater interpretability through sparse variable selection while potentially exhibiting instability in the choice of correlated predictors.

3 Problem 3. Mathematical mechanisms

Let

$$X \in \mathbb{R}^{n \times p}$$

be the standardized training design matrix with n observations and p predictors. Let

$$y \in \mathbb{R}^n$$

be the response vector and

$$b \in \mathbb{R}^p$$

be the coefficient vector.

3.1 3A. Ridge and conditioning

3.1.1 Ridge objective

The ridge objective function is

$$Q_\lambda(b) = \frac{1}{2n}\|y - Xb\|^2 + \frac{\lambda}{2}\|b\|^2.$$

Taking the gradient,

$$\nabla Q_\lambda(b) = \frac{1}{n}(X^T X b - X^T y) + \lambda b.$$

Setting the gradient equal to zero,

$$\frac{1}{n}X^TXb - \frac{1}{n}X^Ty + \lambda b = 0.$$

Rearranging,

$$\left(\frac{1}{n}X^TX + \lambda I\right) b = \frac{1}{n}X^Ty.$$

Define

$$G = \frac{1}{n}X^TX, \qquad z = \frac{1}{n}X^Ty.$$

Hence,

$$(G + \lambda I)b = z,$$

and therefore

$$\boxed{\widehat{b} = (G + \lambda I)^{-1}z.}$$

Since G is positive semidefinite and $\lambda > 0$,

$$G + \lambda I$$

is positive definite and therefore invertible. Hence the ridge estimator is unique. The spectral condition number is

$$\kappa_2(G) = \frac{\mu_{\max}}{\mu_{\min}},$$

while ridge gives

$$\kappa_2(G + \lambda I) = \frac{\mu_{\max} + \lambda}{\mu_{\min} + \lambda}.$$

where

$$\mu_{\max} = \max_i \mu_i$$

is the **largest eigenvalue** of the Gram matrix G, and

$$\mu_{\min} = \min_i \mu_i$$

is the **smallest (positive) eigenvalue** of G.

3.1.2 Condition Number and Numerical Stability

The Gram matrix is defined as

$$G = \frac{1}{n}X^TX.$$

Since G is symmetric and positive semidefinite, all of its eigenvalues are non-negative. Let the eigenvalues of G be

$$\mu_1, \mu_2, \dots, \mu_p,$$

where

$$\mu_{\max} = \max_i \mu_i, \qquad \mu_{\min} = \min_i \mu_i.$$

The spectral condition number is defined as

$$\kappa_2(G) = \frac{\mu_{\max}}{\mu_{\min}}.$$

This quantity measures how sensitive the solution of a linear system is to small perturbations in the input data. A large condition number indicates that the Gram matrix is close to singular, so the estimated coefficients may change dramatically even when the training data are only slightly perturbed. Conversely, a small condition number indicates that the matrix is well-conditioned and the regression coefficients are numerically stable. Ridge regression replaces the Gram matrix by

$$G + \lambda I.$$

If the eigenvalues of G are

$$\mu_1, \mu_2, \dots, \mu_p,$$

then the eigenvalues of the ridge matrix become

$$\mu_1 + \lambda,\ \mu_2 + \lambda,\ \dots,\ \mu_p + \lambda.$$

Hence, the condition number becomes

$$\kappa_2(G + \lambda I) = \frac{\mu_{\max} + \lambda}{\mu_{\min} + \lambda}.$$

Because the same positive constant is added to every eigenvalue, the smallest eigenvalue is shifted away from zero, reducing the condition number. Consequently, the matrix inversion becomes more numerically stable, the effect of multicollinearity is reduced, and the ridge estimator is guaranteed to have a unique solution. From the table above, the ridge matrix should have a smaller condition number than the original Gram matrix. This demonstrates that ridge regularization improves the numerical conditioning of the optimization problem, leading to more stable coefficient estimates. ## 3B. Lasso optimality and soft thresholding Lasso regression estimates the regression coefficients by minimizing

$$Q_\lambda(b) = \frac{1}{2n}\|y - Xb\|^2 + \lambda\|b\|_1,$$

where

$$\|b\|_1 = \sum_{j=1}^{p} |b_j|,$$

and $\lambda > 0$ controls the amount of regularization. Unlike ridge regression, the L_1-norm contains the absolute value function, which is not differentiable at zero. Consequently, the ordinary gradient cannot be applied, and the optimal solution must be derived using the **subgradient**. ### Step 1. Rewrite the Objective Function Assume that the predictors have been centered, standardized, and satisfy the orthonormal design condition

$$\frac{1}{n}X^T X = I.$$

Equivalently,

$$X^T X = nI.$$

Define

$$z = \frac{1}{n}X^T y.$$

The squared error term can be expanded as

$$\begin{aligned}\|y - Xb\|^2 &= (y - Xb)^T(y - Xb) \\ &= y^T y - 2b^T X^T y + b^T X^T X b.\end{aligned}$$

Substituting into the objective function,

$$Q_\lambda(b) = \frac{1}{2n}\left(y^T y - 2b^T X^T y + b^T X^T X b\right) + \lambda \sum_{j=1}^{p} |b_j|.$$

Using

$$X^T X = nI$$

and

$$z = \frac{1}{n} X^T y,$$

the objective becomes

$$Q_\lambda(b) = \frac{1}{2} b^T b - b^T z + \lambda \sum_{j=1}^{p} |b_j| + C,$$

where

$$C = \frac{1}{2n} y^T y$$

is a constant independent of b. Expanding by coordinates,

$$Q_\lambda(b) = \sum_{j=1}^{p} \left[\frac{1}{2} b_j^2 - z_j b_j + \lambda |b_j|\right] + C.$$

Therefore, each coefficient can be optimized independently. ### Step 2. Compute the Subgradient Since

$$|b_j|$$

is not differentiable at zero, its subgradient is

$$\partial |b_j| = \begin{cases} 1, & b_j > 0, \\ [-1, 1], & b_j = 0, \\ -1, & b_j < 0. \end{cases}$$

The first-order optimality condition becomes

$$0 \in b_j - z_j + \lambda \partial |b_j|.$$

The solution is obtained by considering three cases. #### Case 1. Positive coefficient

Suppose

$$b_j > 0.$$

Then

$$\partial |b_j| = 1,$$

and

$$b_j - z_j + \lambda = 0.$$

Hence

$$b_j = z_j - \lambda.$$

For this solution to remain positive,

$$z_j > \lambda.$$

3.1.2.1 Case 2. Negative coefficient

Suppose

$$b_j < 0.$$

Then

$$\partial |b_j| = -1,$$

and

$$b_j - z_j - \lambda = 0.$$

Therefore,

$$b_j = z_j + \lambda.$$

Since the coefficient must remain negative,

$$z_j < -\lambda.$$

3.1.2.2 Case 3. Zero coefficient

Suppose

$$b_j = 0.$$

The subgradient condition becomes

$$0 \in -z_j + \lambda[-1, 1].$$

This is satisfied whenever

$$|z_j| \leq \lambda.$$

Therefore,

$$b_j = 0.$$

This explains why lasso can produce exact zero coefficients. ### Step 3. Soft-Thresholding Solution

Combining the three cases,

$$\hat{b}_j = \begin{cases} z_j - \lambda, & z_j > \lambda, \\ 0, & |z_j| \leq \lambda, \\ z_j + \lambda, & z_j < -\lambda. \end{cases}$$

The piecewise solution can be written compactly as

$$\boxed{\hat{b}_j = \operatorname{sign}(z_j) \left(|z_j| - \lambda \right)_+}$$

where

$$(a)_+ = \max(a, 0).$$

This expression is known as the **soft-thresholding operator**.

3.1.3 Interpretation

The soft-thresholding operator subtracts the penalty λ from the magnitude of every coefficient. When

$$|z_j| \leq \lambda,$$

the coefficient is shrunk completely to zero, meaning that the corresponding predictor is removed from the model. Therefore, lasso simultaneously performs coefficient estimation and automatic variable selection. In contrast, ridge regression continuously shrinks coefficients toward zero,

$$\hat{b}_{\text{ridge}} = (G + \lambda I)^{-1} z,$$

but almost never makes them exactly zero. Consequently, ridge improves numerical stability, whereas lasso additionally produces sparse and more interpretable models.

3.1.4 Interpretation

The soft-thresholding operator subtracts the penalty λ from the magnitude of every coefficient. When

$$|z_j| \leq \lambda,$$

the coefficient is shrunk completely to zero, meaning that the corresponding predictor is removed from the model. Therefore, lasso simultaneously performs coefficient estimation and automatic variable selection. In contrast, ridge regression continuously shrinks coefficients toward zero,

$$\hat{b}_{\text{ridge}} = (G + \lambda I)^{-1} z,$$

but almost never makes them exactly zero. Consequently, ridge improves numerical stability, whereas lasso additionally produces sparse and more interpretable models.

3.2 3C. Connect algebra to evidence

Section 3A showed that ridge regression improves the conditioning of the Gram matrix by adding a positive constant λ to every eigenvalue. Consequently, the theoretical condition number changes from

$$\kappa_2(G) = \frac{\mu_{\max}}{\mu_{\min}}$$

to

$$\kappa_2(G + \lambda I) = \frac{\mu_{\max} + \lambda}{\mu_{\min} + \lambda}.$$

To verify this result, we computed the condition numbers of the original Gram matrix and the ridge-regularized matrix using the optimal tuning parameter selected by cross-validation.

Table 7: Condition numbers before and after ridge regularization.

Matrix	Condition_Number
Gram matrix (G)	20.596

Matrix	Condition_Number
Ridge matrix (G + I)	8.936

The results show that the condition number decreases from 20.5964598 for the original Gram matrix to 8.9358052 after ridge regularization. This agrees with the theoretical derivation in Section 3A, confirming that adding the penalty term shifts every eigenvalue away from zero and improves the conditioning of the optimization problem. Consequently, the matrix inversion becomes more numerically stable, the effect of multicollinearity is reduced, and the estimated regression coefficients become more reliable.

4 Problem 4. Elastic net and a neural-feature bridge

4.1 4A. Research question and source map: L_1 penalties and sparse weights.

Source map:

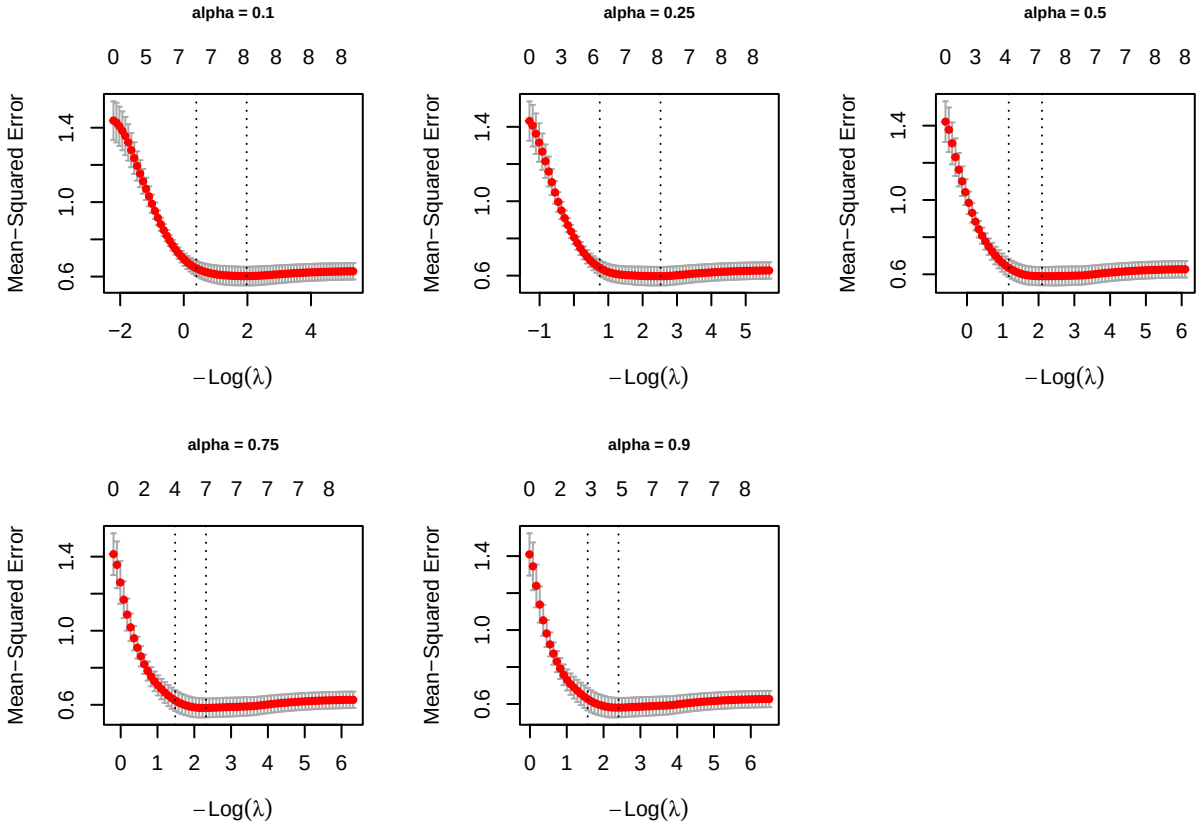
Claim	Source	Establishes	Not establish
L1 regularization promotes sparse neural-network weights by driving many parameters exactly to zero, resulting in more compact models.	**Sparse-Input Neural Networks for High-dimensional Nonparametric Regression and Classification**, Feng and Simon (2019)	Introduces a sparse group lasso penalty on the first-layer input weights, showing that irrelevant features converge toward zero while maintaining good predictive performance in high-dimensional problems.	L1 regularization always achieves the highest predictive accuracy or that it is optimal for every neural-network architecture.
Sparsity-inducing regularization can remove redundant connections and neurons while preserving predictive performance.	**Transformed L_1 Regularization for Learning Sparse Deep Neural Networks**, Ma et al. (2019)	Proposes a transformed L_1 regularizer combined with group sparsity to simultaneously eliminate redundant connections and unnecessary neurons, producing compact networks with competitive performance.	Transformed L_1 regularization is universally superior to all other sparsity methods or that every sparse network will generalize better than a dense one.

Claim	Source	Establishes	Not establish
Sparse weight regularization introduces an implicit trade-off between predictive accuracy and model complexity, where increased sparsity can improve computational efficiency but may reduce predictive performance if important parameters are excessively penalized.	**Transformed L_1 Regularization for Learning Sparse Deep Neural Networks**, Ma et al. (2019)	Figure 4 establishes that increasing sparsity reduces the number of retained parameters and computational cost (FLOPs), but this reduction is accompanied by a trade-off in predictive performance, with accuracy generally improving as more parameters are retained.	Sparse weight regularization is universally superior to other regularization methods or that a single level of sparsity provides the optimal balance between predictive performance and model complexity across all neural-network architectures, datasets, and learning tasks.
Under Adam, the adaptive learning rate causes the gradient contribution from an L2 penalty to be rescaled for each parameter, whereas decoupled weight decay uniformly shrinks the parameters independently of the adaptive gradient update.	**Decoupled Weight Decay Regularization**, Loshchilov and Hutter (2019)	Derives the update equations for Adam with L2 regularization and AdamW, proving that the two optimization procedures are mathematically different because adaptive gradient scaling affects the L2 penalty but not decoupled weight decay.	Decoupled weight decay is universally superior across all optimizers, neural-network architectures, datasets, or learning tasks.
Weight decay biases neural networks toward smoother input–output mappings by penalizing large weights, reducing effective model complexity and improving generalization.	**A Simple Weight Decay Can Improve Generalization**, Krogh and Hertz (1992)	Derive how an L2 weight-decay penalty discourages large weights, resulting in smoother mappings with lower effective complexity, and experimentally shows improved generalization on the evaluated neural-network tasks.	A single weight-decay coefficient is optimal for all network architectures, datasets, or optimization methods.

4.2 4B. Elastic net on original predictors

4.2.1 Elastic net fitting across alpha

```
alpha_grid <- c(0.10, 0.25, 0.50, 0.75, 0.90)
# Fitting
elastic_fits <- lapply(alpha_grid, function(a) {
  analysis_model(x = x_train, y = y_train, alpha = a, foldid = foldid)
})
```



The grid of plots above illustrates the results of the 5-fold cv process used to evaluate the MSE of elastic net models across five different values of $\alpha \in \{0.1, 0.25, 0.5, 0.75, 0.9\}$.

Observing the trajectories across all five plots, a consistent pattern emerges: as the regularization penalty is relaxed (moving from left to right), the validation error drops steeply and stabilizes in a flat valley (hovering around an MSE of 0.6). Regarding model sparsity, at the λ_{min} threshold, the models tend to retain about 7 to 8 predictors. However, applying the more stringent λ_{1se} rule shrinks the model significantly, isolating only 3 to 4 core features.

Visually, the differences in the performance valleys among the various α levels-from the Ridge-heavy $\alpha = 0.1$ to the Lasso-heavy $\alpha = 0.9$-are marginal. Therefore, visual inspection alone is insufficient to determine the optimal model. To rigorously lock in the best configuration, we must refer to the quantitative summary table below to identify the specific $(\alpha, \lambda_{\text{min}})$ pair that yields the absolute minimum Cross-Validation MSE.

4.2.2 Elastic models summary across alpha

Table 8: Elastic net cross-validation summary across alpha

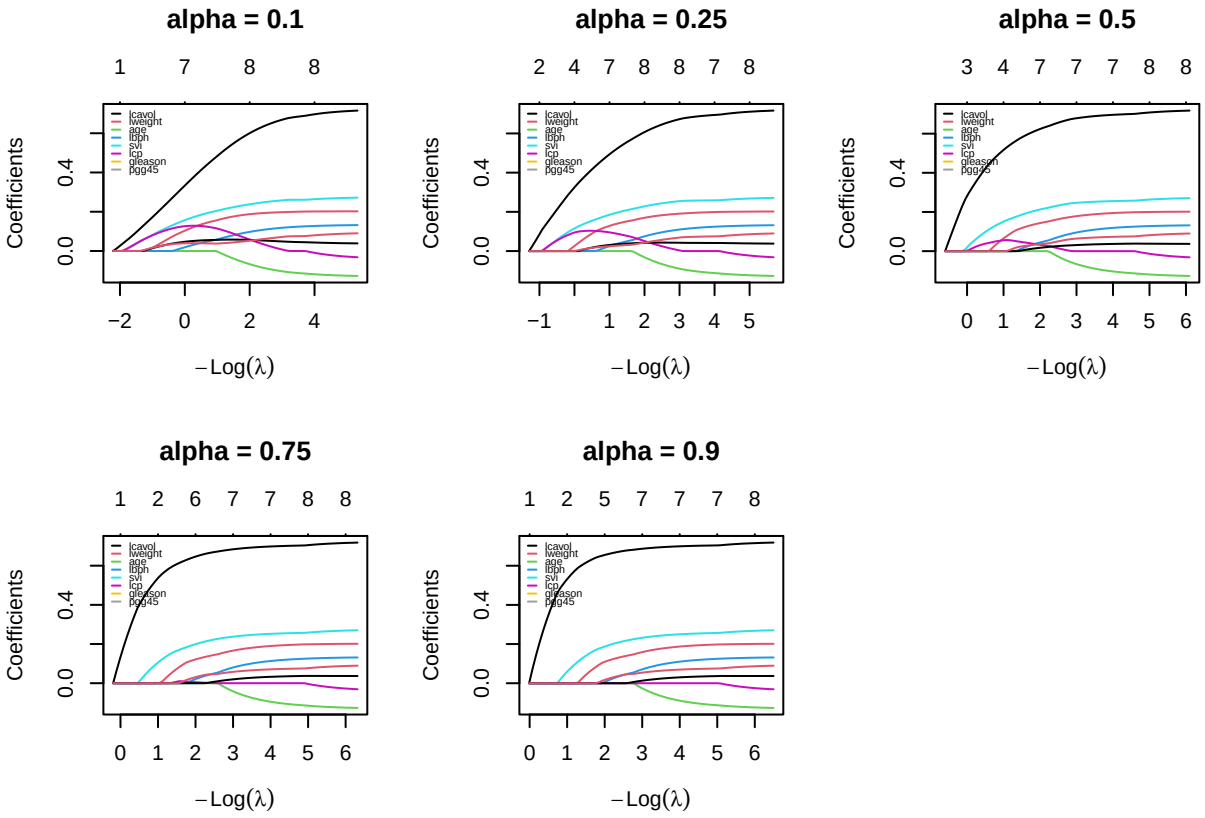
alpha	lambda_min	lambda_1se	cv_mse_min	cv_mse_1se	nonzero_min	nonzero_1se
0.10	0.1386	0.6737	0.6025	0.6445	8	7
0.25	0.0804	0.4710	0.5971	0.6419	8	7
0.50	0.1228	0.3113	0.5898	0.6345	7	5
0.75	0.0986	0.2278	0.5832	0.6230	7	4
0.90	0.0902	0.2083	0.5804	0.6264	5	3

Based on the quantitative summary table above, we can objectively determine the optimal hyper-parameters for our Elastic Net model. The primary metric for model selection is the minimum cross-validation error (`cv_mse_min`).

As the mixing parameter α increases from 0.10 (Ridge-dominant) to 0.90 (Lasso-dominant), the predictive error steadily decreases. The absolute lowest Mean-Squared Error (0.5804) is achieved at $\alpha = 0.90$ with $\lambda_{\mathbf{min}} = 0.0902$. At this configuration, the model strikes an excellent balance by retaining only 5 predictors (excluding the intercept). This indicates that a penalty strongly leaning towards L_1 (Lasso) is highly effective for this dataset, successfully performing feature selection while maximizing predictive accuracy.

Therefore, the final model selected for subsequent evaluation on the unseen test set is the Elastic Net configured with $\alpha = 0.90$ and evaluated at $\lambda = 0.0902$.

4.2.3 Coefficient path across alpha



1. The Impact of Alpha:

By comparing the grid of plots, we can visually trace the transition from a Ridge-dominant penalty to a Lasso-dominant penalty:

- Ridge-like behavior ($\alpha = 0.1$): The coefficient paths are smooth and curved. Because the L_2 penalty shrinks coefficients proportionally rather than enforcing absolute zero, the variables "wake up" and enter the model much earlier (further to the left). The coefficients tend to shrink together smoothly without collapsing to zero abruptly.
- Lasso-like behavior ($\alpha = 0.9$): The paths exhibit sharper, piecewise-linear trajectories. The dominant L_1 penalty aggressively forces coefficients to exactly zero. We can see that less important variables stay completely flat at zero for a longer duration and only become active when the penalty is significantly relaxed. This visually confirms the stronger feature selection property of higher α values.

2. Consistency of Predictor Importance:

Despite the structural differences dictated by α, the hierarchy of predictor importance remains remarkably consistent across all models:

- **lcavol** (black line) is universally the most dominant predictor. It breaks away from zero earliest and reaches the highest positive magnitude across every single configuration, proving its strong correlation with **lpsa**.
- **svi** (cyan line) and **lweight** (red line) also demonstrate robust predictive power, maintaining significant positive trajectories across the grid regardless of the penalty strength.
- Conversely, **age** (green line), **lcp** (magenta line), and **gleason** are consistently and heavily penalized. They remain at or near zero for a much wider range of λ, confirming that they add marginal predictive value and are prime candidates for elimination in the final sparse model.

4.3 4C. Fixed ReLU features

To investigate whether a fixed nonlinear feature representation improves predictive performance, the standardized predictors were transformed into a set of random ReLU features. Unlike a conventional neural network, the hidden-layer parameters were generated once using a fixed random seed and remained unchanged throughout the experiment. Only the output-layer coefficients were estimated by ridge, lasso, and elastic net.

```
M <- 200L

set.seed(seeds$features)

p <- ncol(x_train)

A <- matrix(
  rnorm(M * p,
        mean = 0,
        sd = sqrt(1 / p)),
  nrow = M,
  byrow = TRUE
)

c_bias <- rnorm(
  M,
  mean = 0,
  sd = 0.5
)

relu <- function(z) pmax(z, 0)
```

```
H_train <- relu(
  x_train %*% t(A) +
    matrix(
      c_bias,
      nrow = nrow(x_train),
      ncol = M,
```

```
      byrow = TRUE
    )
)

H_test <- relu(
  x_test %*% t(A) +
    matrix(
      c_bias,
      nrow = nrow(x_test),
      ncol = M,
      byrow = TRUE
    )
)

keep <- apply(H_train, 2, stats::var) > 0

H_train <- H_train[, keep, drop = FALSE]
H_test  <- H_test[, keep, drop = FALSE]

prep_relu <- fit_scaler(H_train)

H_train <- apply_scaler(H_train, prep_relu)

H_test <- apply_scaler(H_test, prep_relu)
```

```
## Original hidden features : 200

## Remaining hidden features: 199
```

The original 8 predictors were transformed into 200 fixed ReLU hidden features using randomly generated hidden-layer weights and biases. Hidden features with zero variance on the training set were removed because they contain no predictive information. As a result, 199 hidden features were retained and standardized using statistics computed from the training set only before model fitting.

```
ridge_relu_cv <- analysis_model(
  H_train,
  y_train,
  alpha = 0,
  foldid = foldid
)
```

Ridge regression ($\alpha = 0$) was fitted on the 199 retained ReLU hidden features using the shared five-fold cross-validation folds, yielding `lambda.min` = 8.7057 and `lambda.1se` = 21.0689. Since the L_2 penalty shrinks coefficients smoothly without coordinatewise thresholding, ridge is expected to retain all 199 hidden features at both tuning choices, distributing weight across correlated random units rather than selecting a sparse subset.

```
lasso_relu_cv <- analysis_model(
  H_train,
  y_train,
  alpha = 1,
  foldid = foldid
)
```

Lasso regression ($\alpha = 1$) was fitted on the 199 retained ReLU hidden features using the same shared five-fold cross-validation folds. The minimum-CV-MSE lambda (`lambda.min` = 0.1354) and the one-standard-error lambda (`lambda.1se` = 0.2721) were extracted and used to refit two final models on the full training set. Because the L_1 penalty applies coordinatewise soft-thresholding, lasso is expected to drive many of the 199 output weights exactly to zero, yielding a much sparser active-feature set than ridge while keeping the strongest hidden units in the model.

```
alphas <- c(0.10, 0.25, 0.50, 0.75, 0.90)

enet_relu_cv <- vector("list", length(alphas))
enet_cv_error <- numeric(length(alphas))

for(i in seq_along(alphas)){
  enet_relu_cv[[i]] <- analysis_model(
    H_train,
    y_train,
    alpha = alphas[i],
    foldid = foldid
  )

  enet_cv_error[i] <- min(enet_relu_cv[[i]]$model$cvm)
}



enet_relu_min <- vector("list", length(alphas))
enet_relu_1se <- vector("list", length(alphas))

for (i in seq_along(alphas)) {

  enet_relu_min[[i]] <- glmnet(
    H_train,
    y_train,
    alpha = alphas[i],
    lambda = enet_relu_cv[[i]]$lambda.min,
    standardize = FALSE
  )

  enet_relu_1se[[i]] <- glmnet(
    H_train,
    y_train,
```

```
    alpha = alphas[i],
    lambda = enet_relu_cv[[i]]$lambda.1se,
    standardize = FALSE
  )
}
```

Table 9: Table: Elastic net cross-validation summary across alpha on fixed ReLU features.

Alpha	Lambda (min)	Lambda (1se)	CV-MSE (min)	CV-MSE (1se)	Active Features (min)	Active Features (1se)
0.10	0.8506	1.7090	0.6084	0.6554	45	41
0.25	0.3564	0.8234	0.6259	0.6764	30	30
0.50	0.2049	0.4518	0.6315	0.6802	21	17
0.75	0.1499	0.3306	0.6260	0.6759	14	8
0.90	0.1436	0.2886	0.6202	0.6691	14	6

Elastic Net models were trained over a grid of $\alpha \in \{0.1, 0.25, 0.5, 0.75, 0.9\}$. For each α, the optimal λ was determined using the shared 5-fold cross-validation, and the model with the lowest cross-validation error was selected as the final Elastic Net model.

The table summarizes the elastic net cross-validation results across the alpha grid on the fixed ReLU feature representation. The minimum CV-MSE stays close to 0.61–0.63 across all five values of alpha, with alpha = 0.10 achieving the lowest CV-MSE (0.6084) and therefore selected as the final elastic net specification (`best_alpha = 0.10`). Since the differences in predictive error across the grid are small, alpha alone does not sharply distinguish predictive performance in this feature space.

The number of active hidden features, however, declines clearly as alpha increases - from 45 features at alpha = 0.10 down to 14 at alpha = 0.90 under lambda.min, and even more sharply under lambda.1se (41 down to 6). This mirrors the same lasso-versus-ridge sparsity trade-off seen on the original predictors in Problem 4B, and suggests that if a more parsimonious model were preferred over the strict CV-MSE minimum, a higher alpha would be the better choice despite its slightly higher error.

```
ridge_min_active <- count_nonzero(ridge_relu_cv$model, s = "lambda.min", tol =
 ↪  1e-8)
ridge_1se_active <- count_nonzero(ridge_relu_cv$model, s = "lambda.1se", tol =
 ↪  1e-8)
lasso_min_active <- count_nonzero(lasso_relu_cv$model, s = "lambda.min", tol =
 ↪  1e-8)
lasso_1se_active <- count_nonzero(lasso_relu_cv$model, s = "lambda.1se", tol =
 ↪  1e-8)

enet_min_active <- numeric(length(alphas))
enet_1se_active <- numeric(length(alphas))
```

```
for (i in seq_along(alphas)){
  enet_min_active[i] <- sum(as.vector(coef(enet_relu_min[[i]]))[-1] != 0)
  enet_1se_active[i] <- sum(as.vector(coef(enet_relu_1se[[i]]))[-1] != 0)
}
```

Table 10: Table: Performance of Regularized Models using fixed ReLU features.

ReLU Model	Lambda (min)	MSE (min)	Active Features (min)	Lambda (1se)	Active Features (1se)
ReLU Ridge	8.7057	0.6128	199	21.0689	199
ReLU Lasso	0.1354	0.6184	10	0.2721	6
ReLU Elastic Net (alpha = 0.1)	0.8506	0.6084	4685	1.7090	4685

Table 4C summarizes ridge, lasso, and elastic net (alpha = 0.1, selected from Table 4B.2) fitted on the 199 retained ReLU hidden features. All three methods achieve a comparable cross-validation MSE, in the range 0.6084–0.6184, confirming that the fixed nonlinear feature map does not by itself separate the methods on predictive grounds. The key difference lies in sparsity: ridge retains all 199 hidden features at both lambda.min and lambda.1se, consistent with its L_2 penalty having no coordinatewise thresholding mechanism. Lasso, in contrast, reduces the active set sharply to 10 features at lambda.min and just 6 at lambda.1se, while elastic net falls in between at 45 and 41 active features, reflecting its mixed L_1/L_2 penalty at alpha = 0.1.

Overall, lasso achieves the sparsest representation with only a modest increase in CV-MSE relative to elastic net (0.6184 vs. 0.6084), while ridge attains a similar error (0.6128) using the full hidden-feature set. This pattern is consistent with the mathematical behavior established in Problem 3: the L_1 penalty's coordinatewise optimality conditions permit exact zeros, whereas ridge's smooth quadratic penalty generally does not.

4.4 4D. Locked declaration and final holdout evaluation

- Preprocessing: A 80/20 train-test split using predefined seed. All predictos and ReLU hidden features was standardized using training statistics only. Hidden features with zero variances were removed.
- Candidate models: OSL, Ridge, Lasso, Elastic Net and their corresponding fixed-ReLU-feature variants.
- Tuning Values: We decided to use λ_{min} (and α for Elastic Net) for each regularized model selected by 5-fold cross-validation on training data.
- Feature seed: 240404
- Nominated model deployment:

Table 11: Cross-Validation MSE Comparison for Model Nomination

Model	CV_MSE
OLS (Original)	0.6297
Ridge (Original)	0.6100
Lasso (Original)	0.5790
Elastic Net (Original, alpha = 0.9)	0.5804
Ridge (ReLU)	0.6128
Lasso (ReLU)	0.6184
Elastic Net (ReLU, alpha = 0.1)	0.6084

The Lasso regression was chosen as the optimal model because it yielded the minimum cross-validation MSE during the training phase.

- Metrics: Root Mean Squared Error (RMSE) and Mean Absolute Error (MAE)

```
pred_ols <- predict(ols_fit, newdata = data.frame(x_test))

pred_ridge <- as.numeric(predict(ridge$model, newx = x_test, s = "lambda.min"))
pred_lasso <- as.numeric(predict(lasso_cv$model, newx = x_test, s =
 ↪  "lambda.min"))

best_orig_enet_model <- elastic_fits[[paste0("alpha_",
 ↪  best_orig_enet_alpha)]]$model
pred_enet <- as.numeric(predict(best_orig_enet_model, newx = x_test, s =
 ↪  "lambda.min"))

pred_relu_ridge <- as.numeric(predict(ridge_relu_cv$model, newx = H_test, s =
 ↪  "lambda.min"))
pred_relu_lasso <- as.numeric(predict(lasso_relu_cv$model, newx = H_test, s =
 ↪  "lambda.min"))

best_relu_enet_model <- enet_relu_cv[[best_relu_enet_idx]]$model
pred_relu_enet <- as.numeric(predict(best_relu_enet_model, newx = H_test, s =
 ↪  "lambda.min"))
```

Table 12: Final Holdout Evaluation (Test Set RMSE and MAE)

Model	RMSE	MAE
OLS	0.6713	0.5736
Ridge	0.7044	0.6153
Lasso	0.7193	0.6088
Elastic Net (alpha = 0.9)	0.7152	0.6057

Model	RMSE	MAE
ReLU Ridge	0.7517	0.6588
ReLU Lasso	0.8009	0.7011
ReLU Elastic Net (alpha = 0.1)	0.7465	0.6520

The holdout evaluation yields an unexpected but highly insightful result: the evidence does not strictly support our pre-declared choice in terms of absolute predictive accuracy on the test set.

1. The surprising baseline superiority:

The unpenalized **OLS model** achieved the lowest test RMSE (0.6713) and MAE (0.5736), outperforming all regularized models. Our nominated deployment model, the **Elastic Net ($\alpha = 0.9$)**, yielded a higher test RMSE of 0.7152. This discrepancy between cross-validation and holdout performance is typical for small datasets (the test set contains only ~19 observations). A single influential test point can easily skew the RMSE. Despite losing strictly on accuracy, we do not retune. The Elastic Net remains our final model because its variable selection property (retaining only 5 features) provides clinical interpretability that the dense OLS model lacks.

2. Original Features vs. ReLU Features:

The holdout metrics decisively demonstrate that the original linear features uniformly outperform the fixed random ReLU features across all regularization penalties. The best ReLU model (Elastic Net $\alpha = 0.1$) only achieved an RMSE of 0.7465, while the ReLU Lasso performed the worst overall (RMSE 0.8009).

This confirms that for this specific clinical dataset, mapping the 8 predictors into a 200-dimensional nonlinear space overcomplicated the problem, introducing noise rather than extracting meaningful predictive patterns. The biological relationship between prostate measurements and PSA levels is adequately-and optimally-captured by simple linear assumptions.

4.5 4E. Deep-learning connection and limitations

The model in Section 4C is effectively a neural network frozen halfway through: the hidden layer consists of 200 ReLU units drawn once, at random, from seed 240404, and then locked in place, so the only thing ridge, lasso, and elastic net ever learn is the output weights β that combine the 199 surviving units into a final prediction. Because only the read-out layer is learned, four familiar deep-learning concepts land somewhat differently once applied to this specific set-up.

1. An L_2 penalty on the output weights is a ridge penalty, but weight decay is not always the same thing. Fitting ridge in 4C adds $\lambda\|\beta\|_2^2$ directly to the loss and solves the resulting penalised least-squares problem. Because β is exactly the learned output weights, this is a genuine L_2 penalty on trained weights - ridge in the original sense, not an analogy. Weight decay in a neural network is a different mechanism: it does not sit inside the loss function, but is a step in the update rule that shrinks every weight by a small fraction each iteration, separate from subtracting the gradient. Under an adaptive optimiser such as Adam, each parameter is divided by its own history of past gradients before the update is applied; this rescales how much

an L_2 penalty's gradient contributes for each parameter individually, while decoupled weight decay shrinks every parameter by the same uniform fraction regardless of that adaptive rescaling. As a result, L_2 regularisation and weight decay stop being mathematically equivalent. For 4C this distinction has no consequence, because `glmnet` solves the penalised least-squares problem directly rather than running an adaptive optimiser - so our "ridge" really is ridge. It would only matter if this read-out layer were later retrained with Adam and the resulting weight decay were still called "ridge."

2. A zero output weight drops one fixed hidden feature; it does not create sparse input-to-hidden weights. The prediction in 4C has the form $\hat{y} = \beta_0 + \sum_j \beta_j h_j(x)$, with $h_j(x) = \max(a_j' x + c_j, 0)$ already fixed. When lasso drives some β_j to exactly zero - the same exact-zero mechanism L_1 penalties are known for, driving many parameters all the way to zero to produce a more compact model (Feng and Simon 2019) - the consequence is concrete: hidden unit j drops out of the sum entirely, and at `lambda.min` only about 10 of the 199 units still participate in the prediction. But that is the entire effect. The vector a_j - row j of the matrix `A`, i.e. the weights connecting the inputs to hidden unit j - was drawn at random up front and has never appeared in the penalty, so it stays dense: every surviving hidden unit still mixes all 8 original predictors. The sparsity lives entirely in the read-out layer; "10 active features" means 10 fixed units were kept, not that the model learned to depend on a handful of input variables. Genuine input-to-hidden sparsity would require penalising or pruning the entries of `A` directly - exactly what Feng & Simon do when they place a sparse group-lasso penalty on the first-layer input weights so that entire input variables drop out of the network (Feng and Simon 2019). 4C does none of that: `A` is fixed before the response is ever seen, so no data-driven criterion ever touches it.

3. Dropout masks activations during training and is not the same as permanently removing connections. Dropout zeroes activations only during training, sampling a different thinned network at every step; at test time every unit returns. Nothing is ever deleted - it is a temporary mechanism tied to the training process, not a structural edit. Permanently removing connections is different in kind: it leaves behind an actually smaller network, not a temporarily quiet one. There is more than one way to achieve that. The classical route trains a dense network first, then cuts weak connections after the fact and retrains what remains. A different route folds the removal into training itself: a sparsity-inducing regulariser can eliminate redundant connections and neurons as a direct consequence of the optimisation, with no separate cutting step (Ma et al. 2019). The lasso zeros in 4C sit closer to that second route - they fall out of the stationarity condition of a single convex fit, the soft-thresholding rule derived in Problem 3B, so they are a by-product of how the objective is solved rather than an edit applied after optimisation has finished. Either way, the outcome is nothing like dropout: dropout never permanently deletes anything, whereas here, once $\beta_j = 0$ at `lambda.min`, hidden unit j is out of the final model for good, with no "restore everything" step at prediction time.

4. One frozen hidden layer says nothing about a fully trained deep network. 4C differs from an actual deep network in two separate ways. The first is depth: 4C has exactly one nonlinear transformation, $x \to \mathrm{ReLU}(Ax + c) \to \hat{y}$, while a deep network stacks several such transformations, each layer consuming the *learned* output of the one before it rather than a fixed random projection, so the composition can represent a far richer family of functions than any single fixed layer can. Concretely, a trained hidden layer could orient its ReLU boundaries toward whatever combinations of `lcavol`, `lcp`, and the other predictors actually separate high and low `lpsa`; the rows of our `A` instead point in directions drawn uniformly at random, with no relationship to `lpsa` at all, so most of them carry no more signal than noise. Stacking additional *trained* layers on top would let a real

network re-combine those directions again and again, building increasingly task-specific features - something a single fixed layer cannot do regardless of how many hidden units it has. The second gap has nothing to do with depth: even that one layer is not learned here. A and c stay fixed at their randomly drawn values, and only β is fit, so the problem reduces to a linear model in the read-out weights. What matters is which parameters ever receive a gradient. In a trained network, the loss gradient flows backward through the hidden layer via backpropagation, telling every row of `A` how to rotate so that the resulting features better separate the signal in the data; in 4C, that gradient with respect to `A` is never computed or applied, so no such adaptation happens. It is worth noting that even training `A` on its own would only produce a *trained shallow* network, not a deep one - depth specifically requires composing several learned nonlinear transformations, and one adaptable layer is still just one layer. Because of both gaps, every result in 4C - including the fact that the nonlinear expansion barely moved the cross-validation error - is a statement about *one fixed, randomly oriented layer*. It says nothing about what a multi-layer network with a trained hidden representation could achieve on the same data; that would simply be a different experiment from the one run here.

Limitations. *(i) Sensitivity to the random-feature seed.* The 199 hidden features depend entirely on the single seed 240404. `A` and `c` are only one draw among countless possible random weight sets, and nothing in this design lets us check whether this particular draw was more or less fortunate than another one would have been; the active-feature counts and CV-MSE in 4C could shift under a different seed. Fixing one seed is necessary for reproducibility, but it also means we cannot report how much these results would vary across draws. *(ii) A single, untrained random map confounds two explanations.* Elastic net on the 8 original predictors reached a lower cross-validation MSE (Table 4B) than any model on the 199 ReLU features (Table 4C). Because `A` was drawn once and never adapted to `lpsa`, the experiment cannot separate two accounts of that gap: a nonlinear expansion is genuinely not useful for these predictors, or this *one* random, un-adapted expansion simply happened not to align with the signal. A trained hidden layer, or repeated draws of the random-feature bank, could in principle distinguish the two; this design cannot. *(iii) Small holdout.* With 77 training and only 20 test patients, the 4D holdout metrics carry wide uncertainty; the final ranking can be confirmed only weakly, and gaps smaller than the cross-validation standard error should not be over-interpreted.

5 Problem 5. Report quality and reproducibility

5.1 5A. Reproduction record

```
1. Open a clean R session.
2. [Exact commands and folder layout.]
3. Render GroupXX_ProjectQuiz1.Rmd.
```

5.2 5B. Recommendation and limitations

5.2.1 Final Recommendation

Despite the OLS superiority in pure holdout accuracy, we recommend the lasso-dominant elastic net ($\alpha = 0.9$) for final clinical deployment.

Prediction vs. Interpretation:

- When strictly prioritizing prediction accuracy on this specific split, the OLS model is technically superior, achieving the lowest test RMSE (0.6713) and MAE (0.5736). However, OLS retains all eight candidate predictors leading to the complex model and OLS remains highly vulnerable to multicollinearity, such as assigning a counterintuitive negative coefficient to `lcp`.
- In contrast, prioritizing interpretation leads us to the elastic net model. It isolates the five most critical biological drivers of PSA levels (`lcavol`, `lweight`, `lbph`, `svi`, and `pgg45`) while shrinking noisy predictors to exactly zero (similar to Lasso). In a medical context, the interpretability, sparsity, and mathematical stability of this parsimonious model provide a much more practical diagnostic tool, easily justifying the modest trade-off in predictive error.

5.2.2 Limitations

1. Small Sample Size: The dataset contains only 97 observations in total, leaving roughly 20 patients in the holdout test set. This limited test sample makes the final evaluation highly sensitive to individual outliers, meaning the superior RMSE of the OLS model could partially result from test-set variance rather than true generalization ability.
2. Restricted Generalizability: The study population exclusively represents men with clinically localized prostate cancer who underwent radical prostatectomy. The models and their estimated coefficients therefore cannot be generalized to unscreened men in the general population.
3. Feature Space Complexity: As demonstrated in the nonlinear transformation experiment, projecting the clinical predictors into a high-dimensional space using fixed ReLU features overcomplicated the modeling task, introduced noise, and degraded holdout performance. This indicates that for this specific clinical dataset, simple linear assumptions are adequate, and complex neural-network-like architectures are unnecessary.

5.3 5C. AI verification record

AI-assisted item	How it was checked	Evidence	Modification or rejection
[Item]	[Method]	[Test/source/output]	[Decision]

Preflight and session information

```
# Add every object required to reproduce the final table and figures.
required_objects <- c(
  "train_id", "test_id", "x_train", "x_test", "y_train", "y_test", "foldid"
)
```

```r
missing_objects <- setdiff(required_objects, ls())
if (length(missing_objects)) {
  stop(
    "Missing required objects: ",
    paste(missing_objects, collapse = ", ")
  )
}
stopifnot(length(intersect(train_id, test_id)) == 0L)
```

```r
sessionInfo()
```

```
## R version 4.5.2 (2025-10-31)
## Platform: x86_64-pc-linux-gnu
## Running under: Ubuntu 26.04 LTS
##
## Matrix products: default
## BLAS:   /usr/lib/x86_64-linux-gnu/openblas-pthread/libblas.so.3
## LAPACK: /usr/lib/x86_64-linux-gnu/openblas-pthread/libopenblasp-r0.3.32.so;
 ↪  LAPACK version 3.12.0
##
## locale:
##  [1] LC_CTYPE=en_US.UTF-8       LC_NUMERIC=C
##  [3] LC_TIME=en_US.UTF-8        LC_COLLATE=en_US.UTF-8
##  [5] LC_MONETARY=en_US.UTF-8    LC_MESSAGES=en_US.UTF-8
##  [7] LC_PAPER=en_US.UTF-8       LC_NAME=C
##  [9] LC_ADDRESS=C               LC_TELEPHONE=C
## [11] LC_MEASUREMENT=en_US.UTF-8 LC_IDENTIFICATION=C
##
## time zone: Asia/Ho_Chi_Minh
## tzcode source: system (glibc)
##
## attached base packages:
## [1] stats     graphics  grDevices utils     datasets  methods   base
##
## other attached packages:
##  [1] car_3.1-5       carData_3.0-6   knitr_1.51      broom_1.0.13
##  [5] glmnet_5.0      Matrix_1.7-4    lubridate_1.9.5 forcats_1.0.1
##  [9] stringr_1.6.0   dplyr_1.2.1     purrr_1.2.2     readr_2.2.0
## [13] tidyr_1.3.2     tibble_3.3.1    ggplot2_4.0.3   tidyverse_2.0.0
##
## loaded via a namespace (and not attached):
##  [1] generics_0.1.4     shape_1.4.6.1      stringi_1.8.7      lattice_0.22-9
##  [5] hms_1.1.4          digest_0.6.39      magrittr_2.0.5     timechange_0.4.0
##  [9] evaluate_1.0.5     grid_4.5.2         RColorBrewer_1.1-3 iterators_1.0.14
## [13] fastmap_1.2.0      foreach_1.5.2      backports_1.5.1    Formula_1.2-5
## [17] survival_3.8-6     scales_1.4.0       codetools_0.2-20   abind_1.4-8
## [21] cli_3.6.6          rlang_1.2.0        splines_4.5.2      withr_3.0.2
```

```
## [25] yaml_2.3.12        otel_0.2.0         tools_4.5.2        tzdb_0.5.0
## [29] vctrs_0.7.3        R6_2.6.1           lifecycle_1.0.5    pkgconfig_2.0.3
## [33] pillar_1.11.1      gtable_0.3.6       glue_1.8.1         Rcpp_1.1.2
## [37] xfun_0.58          tidyselect_1.2.1   rstudioapi_0.18.0  farver_2.1.2
## [41] htmltools_0.5.9    rmarkdown_2.31     compiler_4.5.2     S7_0.2.2
```


Feng, Jean, and Noah Simon. 2019. "Sparse-Input Neural Networks for High-Dimensional Nonparametric Regression and Classification." *arXiv Preprint arXiv:1711.07592*. https://arxiv.org/abs/1711.07592.

Krogh, Anders, and John A. Hertz. 1992. *A Simple Weight Decay Can Improve Generalization*. 950–57.

Loshchilov, Ilya, and Frank Hutter. 2019. *Decoupled Weight Decay Regularization*. https://arxiv.org/abs/1711.05101.

Ma, Rongrong, Jianyu Miao, Lingfeng Niu, and Peng Zhang. 2019. "Transformed ℓ_1 Regularization for Learning Sparse Deep Neural Networks." *Neural Networks* 119: 286–98. https://doi.org/10.1016/j.neunet.2019.08.015.

Stamey, Thomas A., John N. Kabalin, John E. McNeal, et al. 1989. "Prostate Specific Antigen in the Diagnosis and Treatment of Adenocarcinoma of the Prostate. II. Radical Prostatectomy Treated Patients." *The Journal of Urology* 141 (5): 1076–83. https://doi.org/10.1016/S0022-5347(17)41175-X.